TEC | Tecnológico de Costa Rica

Administracion de Tecnologias de Informacion

Bases de Datos
Proyecto #2

Jacqueline Tatiana Solis Cespedes

Elaborado por:
Edwin Mauricio Membreño Artavia - 2024177092
Keilyn Solano Tencio - 2025072273
Ian Fernández Aguilar - 2025099205

.

II Semestre, 2026

Tabla de contenido

Presentación del proyecto	3
Factibilidad técnica económica y operativa	4
Requisitos y reglas del negocio	7
Diseño conceptual	9
Diseño lógico y normalización	10
Diseño físico	12
Implementación de programación y precios	14
Implementación de pruebas y resultados	21
Operación y mantenimiento	30
Conclusiones	32
Evidencias visuales	33
Referencias	38
Anexo de uso de inteligencia artificial en la interfaz	39

BD IMAX

Presentación del proyecto

Este proyecto busca organizar la información de un cine y apoyar la venta de entradas. El sistema permite consultar la cartelera, elegir asientos, pagar de forma simulada y presentar un boleto en la entrada. La principal preocupación es evitar que dos personas compren el mismo asiento y mantener los datos correctos durante todo el proceso.

Construimos una aplicación web con Python y Flask conectada a PostgreSQL. Incluye películas, salas, funciones, precios, reservas, boletos con QR, reembolsos y permisos para cada tipo de usuario. La base integrada tiene 33 tablas y las pruebas guardadas en el repositorio registran 97 casos aprobados.

El programa funciona como un prototipo local. El proyecto también presenta una infraestructura para una operación de mayor tamaño, con servidores distribuidos y copias de seguridad. La propuesta describe una operación mundial y se distingue de la capacidad comprobada en las pruebas locales.

Factibilidad técnica económica y operativa

Dictamen de rechazo

Descartamos la infraestructura inicial porque depende de un solo equipo y no ofrece suficientes controles para una venta de entradas confiable. Además de elegir una base de datos adecuada, necesitamos proteger los asientos, controlar los pagos y recuperar la información cuando ocurra una falla.

Debilidad	Impacto	Respuesta
Access o SQLite en computadora antigua	Dependencia de un solo equipo y capacidad insuficientemente demostrada.	PostgreSQL en infraestructura dedicada, con pruebas de carga y réplica.
Internet residencial de 50 Mbps y Wi Fi	Punto único de falla y límite para tráfico mundial.	Red privada, acceso protegido y enlaces alternativos en sedes críticas.
Reservas escritas en una hoja compartida	No garantiza un único comprador para un asiento.	Inventario por función y asiento, restricciones y transacciones.
Tarjetas y CVV enviados por mensajería	Exposición de información y conciliación manual insegura.	Simulación en el prototipo y proveedor externo evaluado para producción.
Reinicio como recuperación	No recupera pérdidas de disco o borrado lógico.	Réplicas, respaldos separados y ejercicios de restauración.
Recursos sin dimensionamiento suficiente	Costos y capacidad sin relación demostrada con la demanda.	Supuestos explícitos, medición y proyección de almacenamiento y operación.

Arquitectura propuesta

Para atender varios países se propone distribuir la aplicación entre regiones. Cada función tendría un servidor responsable de confirmar sus reservas y ventas. Las consultas de cartelera podrían repartirse entre servidores. La confirmación de un asiento consulta la información actual de la base de datos.

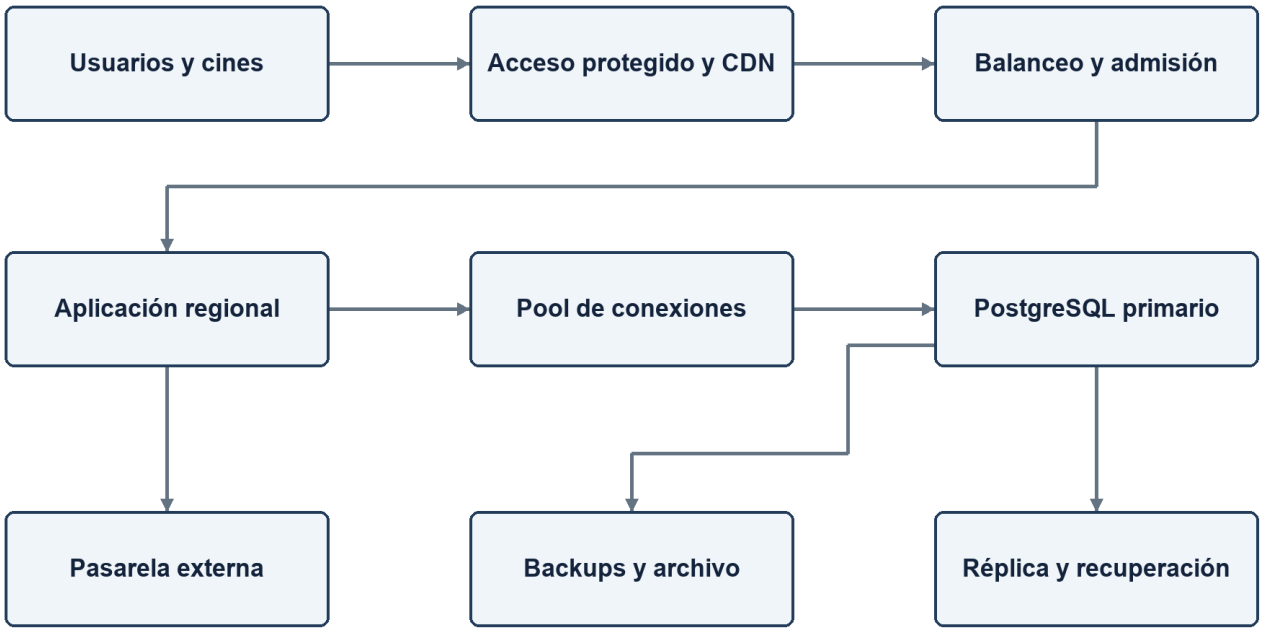


Figura 1 Arquitectura global propuesta para la operación futura

La propuesta incluye repartir las solicitudes entre varios servidores y limitar las conexiones abiertas a PostgreSQL. Una copia de la base ayudaría a continuar el servicio ante una falla y los respaldos permitirían recuperar información anterior. El diagrama representa una propuesta de crecimiento y no una infraestructura ya instalada.

Riesgos y medidas

Riesgo	Medida y estado
Caída de base o región	Réplica y respaldo remoto para la restauración y la continuidad del servicio.
Doble venta o QR repetido	Bloqueos, restricciones y transición de estado comprobados con 50 solicitudes simultáneas.
Respuesta tardía del pago	Referencia de pago e idempotencia comprobadas mediante pagos simulados.
Saturación o DDoS	Caché, admisión, límites de conexiones y filtrado como parte de la arquitectura propuesta.
Abuso de permisos o pérdida de historia	Roles, motivos y bitácoras comprobados; separación de propietarios y archivo inmutable propuestos.

Factibilidad económica y riesgos

Escenario económico preliminar

Comparamos una estimación de AWS y Azure para tres años. El escenario considera tres servidores de base de datos, cuatro para la aplicación y dos para manejar las conexiones. Los servidores de base de datos tendrían 16 procesadores virtuales, 64 GiB de memoria y 512 GiB para datos. Los de aplicación tendrían 4 procesadores virtuales y 16 GiB de memoria, y los de conexiones tendrían 2 y 8 GiB respectivamente. Cada servidor contempla además 64 GiB de arranque.

El cálculo supone servicio continuo durante 730 horas al mes. Incluye USD 20 000 de inversión inicial, USD 3 200 mensuales de operación y una reserva inicial de USD 1 200 para consumos variables. A la infraestructura y esa reserva se les agrega un 20 % para imprevistos. PostgreSQL no añade en este cálculo una licencia comercial por servidor, aunque su administración sí genera costos.

Escenario	AWS en USD	Azure en USD
Sin crecimiento	297 996,86	299 738,09
Crecimiento anual de 10 %	303 417,05	305 146,10
Crecimiento y reserva variable duplicada	360 613,85	362 342,90

Los montos corresponden a un escenario de planificación y representan costos de referencia. El almacenamiento, el procesamiento y la red influyen en el costo de una operación de mayor tamaño. La diferencia entre los dos proveedores es pequeña dentro de este escenario.

El costo total suma la inversión inicial y los gastos de los 36 meses. No cubre los años cuarto y quinto de conservación de datos. Tampoco incluye todos los costos de pasarela de pagos, impuestos, certificaciones, conectividad de los cines y desarrollo del producto.

Retorno de inversión y operación

Para estimar el retorno de inversión restamos el costo al beneficio acumulado y dividimos el resultado entre el costo. Si cada compra adicional recuperada deja USD 3 y se recupera un 0,05 % adicional de compras, el resultado preliminar es 17,82 % para AWS y 17,15 % para Azure. Son ejemplos basados en supuestos y no beneficios ya comprobados.

La operación necesita personal para administrar la base, mantener los servidores, atender problemas y trabajar en taquilla y puerta. El presupuesto contempla 80 horas mensuales de soporte, con un alcance distinto de la atención permanente durante todo el día.

Requisitos y reglas del negocio

Procesos y actores

El trabajo se organiza en programación de funciones, venta de entradas, ingreso a sala, reembolsos y administración. El cliente consulta y compra. El taquillero atiende ventas presenciales. El operador de puerta valida boletos. El contador consulta reportes y el administrador configura el sistema y autoriza cambios especiales.

El sistema debe permitir buscar películas, elegir asientos, aplicar tarifas y promociones, reservar, simular pagos y emitir boletos. También debe impedir ventas duplicadas, proteger el acceso y registrar los cambios importantes. Las pruebas locales revisan estas reglas, mientras que la capacidad para millones de usuarios requiere otro tipo de ensayo.

Políticas adoptadas

Política	Decisión utilizada
Selección	De 1 a 6 asientos distintos por compra y una función por compra.
Reserva	120 segundos medidos por PostgreSQL; no se renueva al recargar.
Distribución	Se rechazan cambios incompatibles con reservas o boletos vigentes.
Limpieza	15 minutos entre fin de función e inicio de la siguiente en la sala.
Ingreso	Desde 30 minutos antes hasta 15 después del inicio.
Devolución ordinaria	Hasta 2 horas antes del inicio y para boletos sin usar.
Excepción	Reembolso autorizado por administrador y con motivo.
Mapa	Consulta de disponibilidad cada 2 segundos y revalidación al reservar.
Moneda y cargo	USD y cargo de servicio inicial de USD 0.
Promoción inicial	Pareja General 2x1 los miércoles, Adulto y General.
Auditoría	Conservación mínima propuesta de 5 años.

Los valores que la consigna presenta como ejemplos se concretaron en los acuerdos del grupo. La promoción del miércoles depende del día en que comienza la función según la hora del cine. No depende del día en que el cliente realiza la compra.

Las reglas del negocio se reflejan en las tablas, las funciones y los casos de prueba. Los permisos se consultan en la base de datos. Ocultar un botón no sería suficiente para impedir una operación no autorizada.

Dimensionamiento de datos

Datos del escenario

La consigna plantea 8 millones de usuarios conectados al mismo tiempo y un pico de 50 000 transacciones por segundo durante 10 minutos. Eso representa hasta 30 millones de operaciones durante el pico, aunque no todas serían compras. El cálculo toma esos 10 minutos como referencia para el escenario de mayor demanda.

También se indican 500 funciones diarias, 1 500 salas, 15 millones de boletos mensuales y cinco años de auditoría. Con 30 días se obtienen 15 000 funciones al mes y un promedio de 1 000 boletos por función. Estas cifras son supuestos de dimensionamiento para la operación propuesta.

Supuestos y magnitudes

El cálculo inicial supone 2,5 boletos por compra y que el 60 % de las reservas termina en pago. Así se estiman 6 millones de compras pagadas y 10 millones de intentos de reserva al mes. Como cada reserva guarda un asiento, esos intentos podrían producir unos 25 millones de filas de reserva. Una compra puede incluir varios asientos y, por tanto, varias filas.

La propuesta conserva seis meses de información de uso frecuente, dos copias de la base para continuidad, dos copias del archivo histórico y respaldos de 30 días. Para estimar espacio se supone que el archivo puede reducirse cuatro veces mediante compresión y los respaldos dos veces. El registro de cambios de PostgreSQL se calcula con un factor de 2,5. Son factores de estimación para este escenario.

Componente al mes 60	Sin crecimiento	Con crecimiento
Primario activo	220,8	323,3
Dos réplicas	441,6	646,6
Archivo con dos copias	530,9	648,3
Backups	603,6	883,7
Total aproximado en TiB	1,75	2,44

La tabla presenta los valores de referencia utilizados para estimar almacenamiento. El cálculo contempla una copia de los respaldos. Una segunda copia representa almacenamiento y costo adicionales dentro de la propuesta de continuidad.

Método de cálculo

El espacio se calcula a partir de la cantidad de registros esperada y el tamaño de cada fila. Luego se suman los índices, las copias, el historial y los respaldos. Los intentos fallidos también generan registros de auditoría. Las pruebas con pocos usuarios sirven para comprobar reglas, pero no permiten calcular directamente el rendimiento mundial.

Diseño conceptual

Entidades y relaciones

El catálogo reúne cines, salas, zonas, asientos, películas y funciones. Una tarifa indica el precio de una entrada según la función, la zona y la categoría del cliente. Las promociones guardan las condiciones necesarias para aplicar un descuento.

Cada compra tiene un detalle con los asientos y sus precios. El inventario indica cuáles están disponibles para cada función. Una reserva los aparta durante un tiempo limitado y un pago aprobado permite emitir los boletos.

Los usuarios y sus roles determinan quién puede realizar cada operación. Las bitácoras guardan los intentos de ingreso y los cambios administrativos. Las solicitudes de reembolso permiten devolver todos los boletos de una compra o solo una parte.

Cardinalidades y decisiones

Un cine tiene varias salas, cada sala tiene zonas y cada zona contiene asientos. Una película puede tener varias funciones, pero cada función se realiza en una sola sala. Una película puede pertenecer a varios géneros y una función puede tener varias promociones. Estas relaciones se organizan mediante tablas intermedias.

Una compra puede incluir varios asientos y boletos. El mismo asiento físico puede venderse en funciones diferentes. Por eso su disponibilidad se guarda para cada función y no como un único estado permanente del asiento.

La capacidad de una sala se obtiene contando sus asientos. Los géneros y las condiciones de promoción se guardan en tablas separadas para poder consultarlos y validarlos sin usar listas dentro de una sola columna.

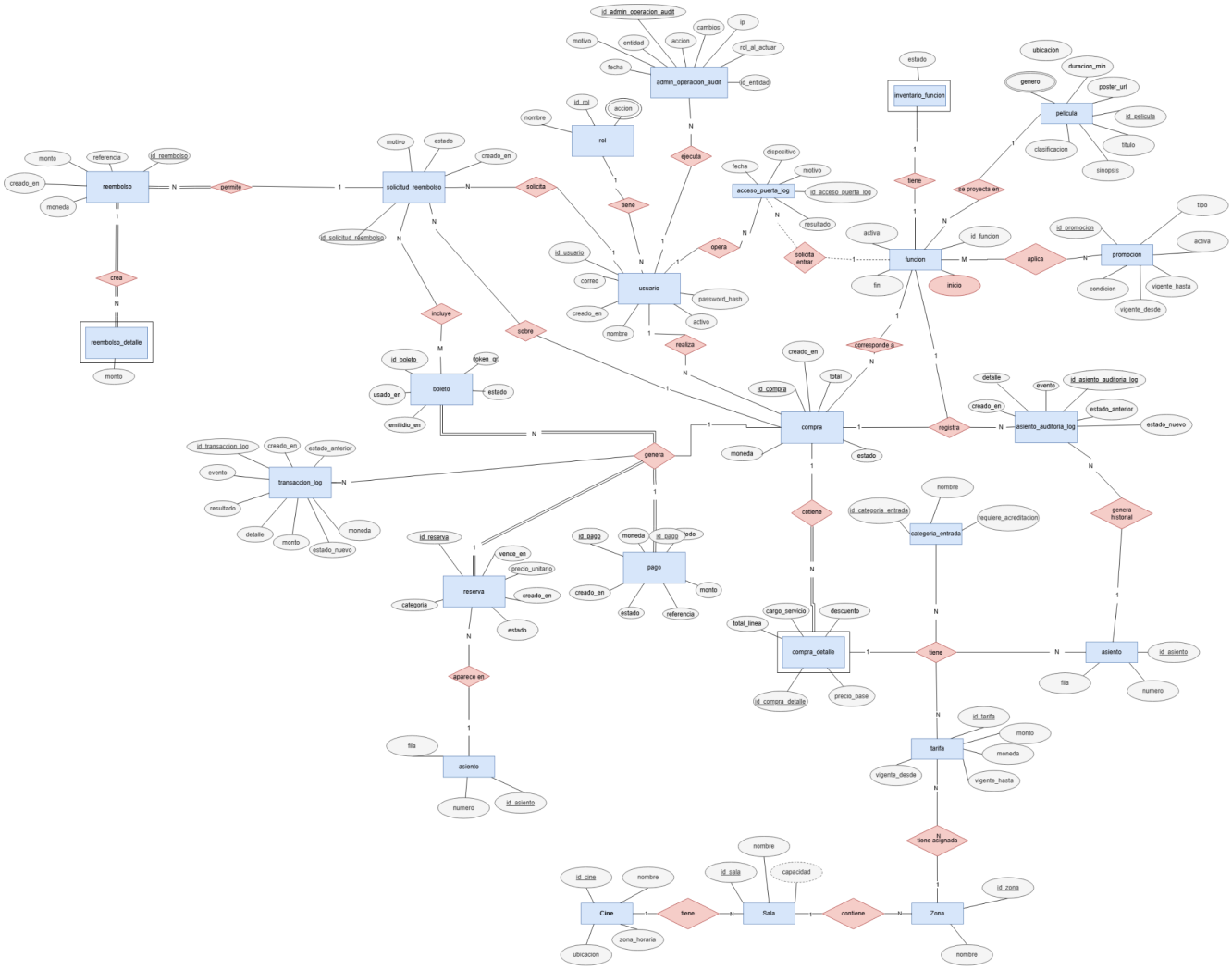


Figura 2 Vista general del diagrama conceptual en notación de Chen

El diagrama relaciona la distribución del cine con la programación y la venta. Los cines se dividen en salas, zonas y asientos. Las películas se ofrecen en funciones y sus precios pueden cambiar según la categoría y las promociones disponibles.

Durante la compra se apartan los asientos y se guarda el detalle de la operación. El pago se procesa de forma simulada. Solo cuando se aprueba se emiten los boletos con sus códigos QR.

Al ingresar, la base verifica el boleto y registra el resultado en la bitácora de puerta. El modelo también contempla reembolsos y anulaciones para conservar el historial de lo ocurrido.

Diseño lógico y normalización

El modelo lógico convierte las entidades en tablas. Cada tabla tiene una clave primaria para identificar sus filas. Las claves foráneas conectan las tablas y las relaciones de muchos a muchos se resuelven con tablas intermedias.

La normalización ayuda a organizar los datos sin repetir información innecesaria. Guardamos los géneros y los permisos en filas separadas, hacemos que los datos dependan de su clave completa y mantenemos los nombres de cines, películas y roles en sus propias tablas.

Claves y conservación de información

En inventario_funcion la función y el asiento forman la clave primaria. Esa combinación permite saber si una ubicación está disponible para una función específica. Las restricciones evitan dos ocupaciones vigentes del mismo asiento y mantienen las referencias entre una compra y su función.

El detalle de compra guarda el precio y el descuento que se aceptaron al reservar. Si después cambia una tarifa, la compra anterior conserva su importe. La auditoría también guarda el rol que tenía la persona cuando hizo el cambio, aunque su rol se modifique más adelante.

Estas copias históricas son necesarias para explicar operaciones pasadas. Se mantienen junto con las referencias al catálogo y se revisan mediante restricciones y funciones para evitar diferencias entre los datos relacionados.

Dependencias representativas

Relación	Dependencia o clave relevante
pelicula	id_pelicula determina título, sinopsis, clasificación, duración y póster.
sala	id_sala determina cine y nombre; cine y nombre constituyen clave alternativa.
zona	id_zona determina sala y nombre; sala y nombre son únicos.
asiento	id_asiento determina zona, fila y número; la ubicación es única dentro de la zona.
funcion	id_funcion determina película, sala, inicio, fin y actividad.
compra_detalle	Su identificador determina asiento, categoría y cotización histórica.
inventario_funcion	Función y asiento identifican una sola fila de disponibilidad.
rol_permiso	Rol y acción identifican un permiso sin duplicarlo.

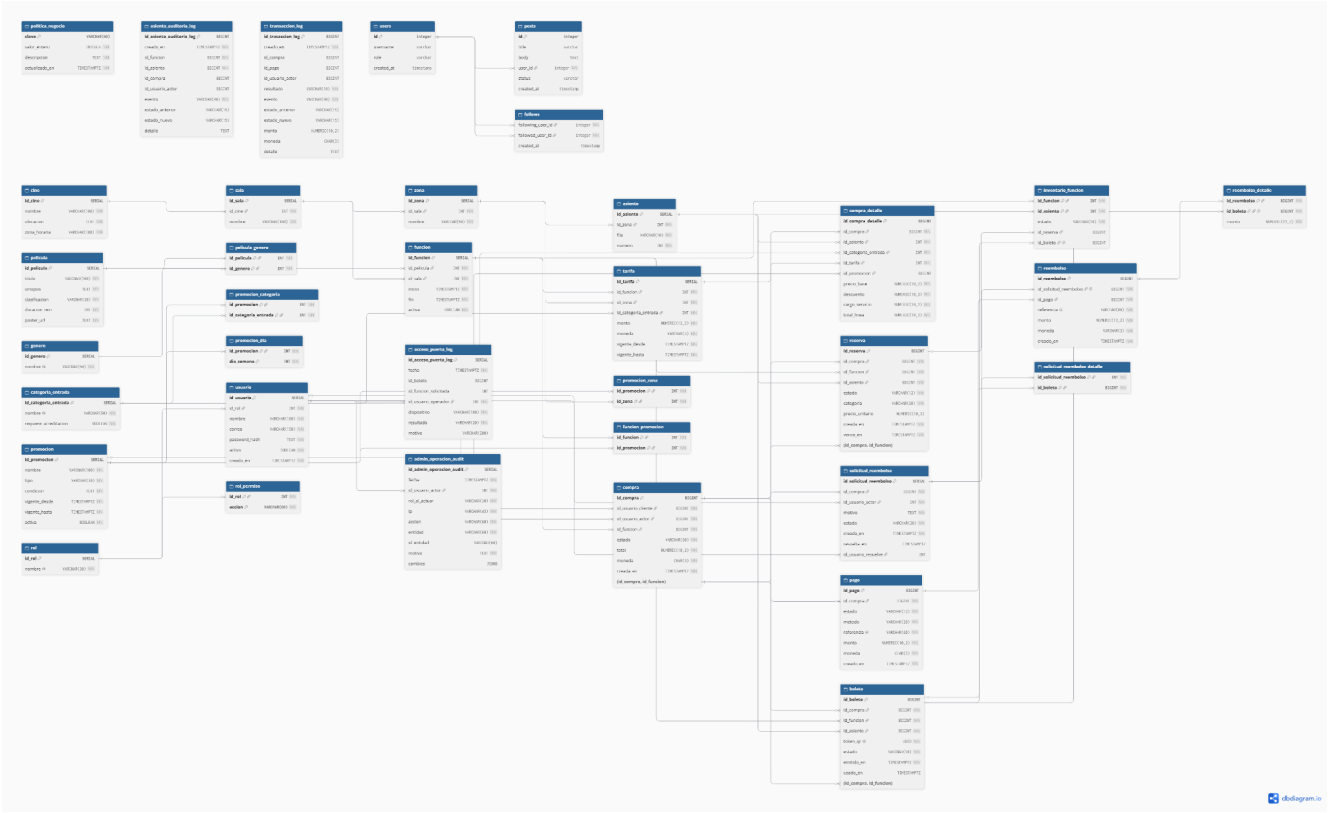


Figura 3 Diagrama relacional

El diagrama relacional muestra cómo las tablas implementan el funcionamiento del cine. Las claves foráneas conectan sus datos y tablas como pelicula_genero y funcion_promocion resuelven relaciones de muchos a muchos.

Diseño físico

Elegimos PostgreSQL 18.6 para implementar la base. Usamos NUMERIC para los importes y fechas con zona horaria para programar funciones. Los catálogos usan identificadores INT generados con SERIAL. Las tablas de ventas conservan identificadores BIGINT para sus registros.

El esquema de referencia tenía 50 índices, incluidos los que PostgreSQL crea para claves primarias y valores únicos. Su función inicial es identificar registros y proteger la integridad. Más adelante se comparan los planes antes y después de agregar índices de apoyo para las consultas.

Restricciones y programación en la base

Integridad estructural

Las claves primarias identifican cada registro y las foráneas exigen que sus referencias existan. UNIQUE evita datos repetidos y CHECK limita los valores permitidos. Las funciones y los triggers completan las reglas que requieren revisar varias filas o tablas.

Por ejemplo, una tarifa solo puede usar una zona de la sala donde se realiza la función. Sus periodos de vigencia no deben cruzarse para la misma combinación. Al programar, la base revisa los horarios y respeta el tiempo de limpieza entre funciones.

Objetos relevantes

Objeto o mecanismo	Finalidad
validar_horario_funcion	Rechazar cruces de horario y limpieza, incluso con operaciones simultáneas.
validar_vigencia_tarifa	Evitar más de una tarifa válida para la misma combinación e instante.
preparar_inventario	Crear filas faltantes sin reiniciar asientos reservados o vendidos.
reservar_asientos	Validar función, asientos, categoría y precio y guardar reserva y cotización.
procesar_pago	Revalidar compra y vencimiento, confirmar o liberar y emitir boletos tras éxito.
liberar_reservas_vencidas	Expirar reservas sin depender de que el navegador siga abierto.
validar_y_consumir_ingreso_qr	Comprobar función, ventana, estado y acreditación y aceptar un solo ingreso.
solicitar_reembolso y resolver_reembolso	Aplicar política y autorización de la devolución.
revertir_venta	Ejecutar devolución, anulación y liberación conservando la historia.
guardia_rbac y auditoría	Verificar permisos y registrar cambios críticos.
Protección por sala	Coordinar distribución con inventario y actividad de ventas.

Bloqueos y aislamiento

Usamos el nivel READ COMMITTED, que permite consultar los cambios ya confirmados por otras operaciones. Antes de modificar una reserva, una compra o un boleto, la base bloquea los registros necesarios. Así dos operaciones no pueden cambiar el mismo estado sin revisarlo. Si una operación falla, se revierte. Un reintento de pago conserva su referencia para evitar duplicados.

Python revisa los formularios y PostgreSQL vuelve a comprobar las reglas al guardar. El cambio y su registro de auditoría se confirman juntos. Si no puede guardarse la auditoría, la operación tampoco queda completada.

Implementación de programación y precios

Catálogo salas y funciones

Los formularios permiten registrar y editar películas con título, sinopsis, clasificación, duración y póster. Python revisa los datos y los permisos del usuario antes de enviarlos a la base.

Los asientos se pueden crear por filas para evitar ingresarlos uno por uno. Si uno de los números ya existe, se cancela el lote completo. La capacidad se calcula con los asientos que realmente quedaron guardados.

La programación usa la hora local del cine y calcula la salida con la duración de la película. La base rechaza cruces de horario y respeta la limpieza. Una función inactiva no ocupa horario, pero al activarla se revisan las reglas otra vez.

Tarifas y promociones

Cada tarifa corresponde a una función, zona y categoría, y tiene un periodo de vigencia. La compra necesita encontrar una tarifa válida. Si falta, no se inventa un precio ni se continúa con un importe incorrecto.

La promoción Pareja General aplica los miércoles para entradas Adulto en zona General. Con un precio de USD 8, dos entradas elegibles cuestan USD 8 y tres cuestan USD 16. La entrada que queda sin pareja paga el precio completo.

Al reservar quedan guardados el precio y el descuento. Cambiar después la tarifa no modifica las compras anteriores. El detalle permite explicar cómo se obtuvo el total.

Estados del negocio

Entidad	Estados utilizados
Compra	Pendiente, Pagada, Cancelada, Expirada, ParcialmenteReembolsada, Reembolsada.
Reserva	activa, confirmada, expirada, liberada.
Boleto	Emitido, Usado, Anulado.
Solicitud de reembolso	pendiente, ejecutada, rechazada.

Estos estados permiten reconocer qué pasó con cada operación. Una compra pagada no es lo mismo que una reserva vencida o un boleto utilizado.

Reserva pago y entrega digital

Reserva de asientos

El usuario elige la función, la categoría y los asientos. La base revisa que la función esté activa, que los asientos pertenezcan a su sala y que la selección tenga entre uno y seis asientos distintos. Si todo está disponible, guarda la compra y la reserva en una misma operación.

Si uno de los asientos ya está ocupado, se rechaza toda la selección. El precio lo calcula el sistema con las tarifas y promociones guardadas. No se acepta un total decidido desde el navegador.

Vencimiento y pago

Los asientos se apartan durante 120 segundos según el reloj de la base de datos. Recargar la página no amplía ese tiempo. El servidor revisa los vencimientos cada dos segundos y las operaciones de pago también comprueban que la reserva siga vigente.

Un pago rechazado cancela la compra y libera los asientos. Un pago aprobado a tiempo emite los boletos. Repetir la misma confirmación no debe generar otros boletos ni otro cobro. Un pago tardío no recupera un asiento que ya se liberó.

En este proyecto el pago es simulado. Para una versión con pagos reales sería necesario conectar un proveedor externo y revisar los casos en que un cobro y la confirmación de la compra no llegan al mismo tiempo.

Confirmación y consulta

El cliente puede consultar sus boletos después de aprobar el pago. El taquillero consulta los de la venta que atendió. El sistema revisa los permisos y la pertenencia de la compra para que una persona no pueda ver el QR de otra.

Las evidencias visuales al final del contenido muestran la cartelera, la selección de asientos, el temporizador y los boletos emitidos.

Ingreso reembolsos y permisos

Control en puerta

El operador elige la función e ingresa el código del boleto. La base revisa que corresponda a esa función, que siga válido y que esté dentro del horario permitido. Si la categoría requiere acreditación, el operador debe comprobarla. El primer ingreso se acepta y el siguiente se rechaza.

El ingreso se permite desde 30 minutos antes hasta 15 minutos después del inicio. Un boleto puede estar vendido para una función futura y todavía no permitir la entrada. Los intentos rechazados también quedan registrados.

Reembolsos y ajustes

El cliente solicita el reembolso y el administrador revisa si cumple la política. Una excepción necesita un motivo. Si la devolución es parcial, solo se anulan los boletos elegidos y se liberan esos asientos. Un boleto ya utilizado no vuelve a estar disponible como si no se hubiera usado.

Las cortesías y los cambios de asiento también usan el inventario y la auditoría. Si un ingreso y una devolución intentan modificar el mismo boleto al mismo tiempo, la base permite únicamente el cambio que todavía sea válido.

Permisos por rol

Rol	Acciones relevantes habilitadas
Cliente	Compra propia, boletos propios y solicitud de reembolso.
Taquillero	Venta presencial para cliente y consulta de la venta atendida.
Operador de puerta	Validación de ingreso.
Contador	Tarifas de lectura, reportes y auditoría.
Administrador	Catálogo, tarifas, políticas, resolución y excepción de reembolso, cortesías y cambio de asiento.

La base contiene 28 asociaciones entre roles y acciones. Cada perfil tiene sus propias tareas. Un administrador no recibe automáticamente los permisos de compra de un cliente ni los de un operador de puerta. El permiso de administrar usuarios existe en la base, aunque el prototipo no incluye una pantalla completa para esa tarea.

La sesión identifica al usuario y el servidor revisa sus permisos en cada solicitud. Los formularios incluyen un control contra envíos no autorizados. Las bitácoras impiden cambios y borrados ordinarios, aunque una cuenta propietaria o con permisos máximos necesita controles adicionales.

Consultas analíticas

Ocupación y recaudación por función

Este reporte compara los asientos vendidos con la capacidad de la sala y muestra el dinero cobrado, devuelto y neto. Los boletos anulados no cuentan como ocupación. Las funciones sin ventas también aparecen con valores en cero.

En una prueba de devolución parcial se conservaron USD 16 de ingreso bruto, USD 8 devueltos y USD 8 netos. Quedó un boleto vigente. Ese resultado pertenece a una prueba de integración y es distinto de la captura posterior de una función sin ventas.

Top de películas y métodos de pago

El Top 5 cuenta los boletos emitidos de los últimos siete días y agrupa el resultado por película. No cuenta los anulados. Una cortesía puede aumentar la cantidad de boletos aunque no genere ingresos.

El reporte de métodos de pago muestra cuántas compras se pagaron con cada método y qué porcentaje representan. También compara los montos. Los pagos rechazados no deben aumentar esas cifras.

Auditoría por administrador

La consulta de auditoría agrupa las acciones por administrador durante los últimos 30 días. Permite revisar cambios manuales, cortesías, reembolsos e intentos rechazados. Una segunda consulta muestra el detalle de cada acción.

A continuación se incluyen las consultas SQL y sus resultados. El acceso a estos reportes depende de los permisos de la cuenta. El nombre R identifica el reporte de negocio y el nombre Q identifica su consulta de evaluación de rendimiento.

Reporte 1: Ocupación y recaudación por función

SQL

```
-- R1. Ocupación y recaudación por función.
WITH param AS (SELECT NULL::int AS id_funcion),
capacidad AS ( -- asientos físicos de cada sala (se calcula, no se guarda)
 SELECT z.id_sala, count(a.id_asiento) AS asientos
 FROM zona z JOIN asiento a ON a.id_zona = z.id_zona
 GROUP BY z.id_sala
),
ventas AS ( -- PREAGREGADO por función ANTES de unir (evita multiplicar importes)
 SELECT c.id_funcion,
 count(b.id_boleto) FILTER (WHERE b.estado IN ('Emitido','Usado'))  AS vendidos,
 count(b.id_boleto) FILTER (WHERE b.estado = 'Usado') AS usados,
 coalesce(sum(cd.total_linea), 0) AS bruto,
 coalesce(sum(cd.total_linea) FILTER (WHERE b.estado = 'Anulado'),0) AS devuelto
 FROM compra c
 JOIN compra_detalle cd ON cd.id_compra = c.id_compra
 LEFT JOIN boleto b ON b.id_compra = cd.id_compra AND b.id_asiento = cd.id_asiento
 WHERE c.estado IN ('Pagada','ParcialmenteReembolsada','Reembolsada') -- solo ventas cobradas
 GROUP BY c.id_funcion
)
SELECT f.id_funcion, p.titulo AS pelicula, ci.nombre AS cine, s.nombre AS sala, f.inicio,
 coalesce(cap.asientos,0) AS capacidad,
 coalesce(v.vendidos,0) AS boletos_vendidos,
 round(100.0 * coalesce(v.vendidos,0) / nullif(cap.asientos,0), 2) AS pct_ocupacion,
 coalesce(v.bruto,0) AS ingreso_bruto,
 coalesce(v.devuelto,0) AS total_devuelto,
 coalesce(v.bruto,0) - coalesce(v.devuelto,0) AS ingreso_neto
FROM funcion f
```

```
JOIN pelicula p ON p.id_pelicula = f.id_pelicula
JOIN sala s ON s.id_sala = f.id_sala
JOIN cine ci ON ci.id_cine = s.id_cine
CROSS JOIN param
LEFT JOIN capacidad cap ON cap.id_sala = f.id_sala -- LEFT: funciones sin ventas salen con 0 %
LEFT JOIN ventas v ON v.id_funcion = f.id_funcion
WHERE param.id_funcion IS NULL OR f.id_funcion = param.id_funcion
ORDER BY f.inicio DESC, f.id_funcion;
```

Explicación del resultado

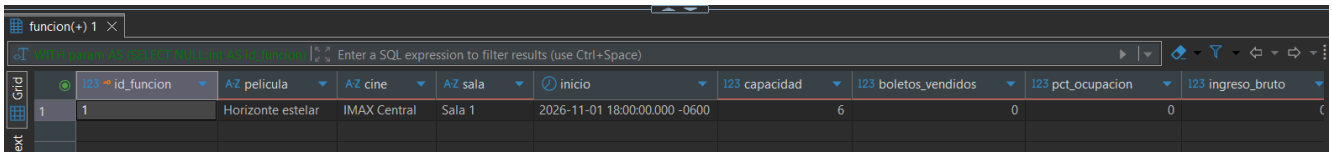
La ocupación se obtiene al dividir los boletos vigentes entre la capacidad y multiplicar por cien.

El ingreso neto es el monto cobrado menos lo devuelto. La devolución se resta una sola vez.

Primero se suman las ventas de cada función y después se unen los demás datos. Esto evita contar el mismo importe varias veces.

Las funciones sin ventas se conservan en el resultado con ocupación e ingresos en cero.

Salida obtenida. La función mostrada tiene seis asientos y ningún boleto vendido.



Reporte 2: Top 5 películas de la última semana

SQL

```
-- R2. Top 5 películas por boletos vendidos en los últimos 7 días (168 h móviles).
WITH corte AS (SELECT now() AS hasta, now() - interval '168 hours' AS desde),
por_pelicula AS (
 SELECT f.id_pelicula,
 count(b.id_boleto)  AS vendidos,
 sum(cd.total_linea) AS recaudado
 FROM boleto b
 JOIN compra_detalle cd ON cd.id_compra = b.id_compra AND cd.id_asiento = b.id_asiento
 JOIN compra c ON c.id_compra = cd.id_compra
 JOIN funcion f ON f.id_funcion = c.id_funcion
 CROSS JOIN corte k
 WHERE b.emitido_en >= k.desde AND b.emitido_en < k.hasta -- intervalo [desde, hasta)
 AND b.estado IN ('Emitido','Usado') -- excluye anulados
 AND c.estado IN ('Pagada','ParcialmenteReembolsada')
 GROUP BY f.id_pelicula
)
SELECT row_number() OVER (ORDER BY pp.vendidos DESC, pe.id_pelicula) AS posicion,
 pe.id_pelicula, pe.titulo, pe.clasificacion,
 pp.vendidos AS boletos_vendidos, pp.recaudado
FROM por_pelicula pp
JOIN pelicula pe ON pe.id_pelicula = pp.id_pelicula
ORDER BY pp.vendidos DESC, pe.id_pelicula -- desempate determinista: menor id_pelicula
LIMIT 5;
```

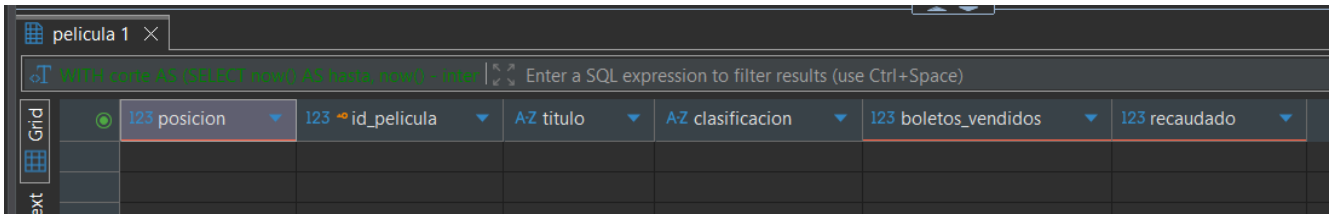
Explicación del resultado

El periodo se calcula con la fecha de emisión del boleto y el mismo instante de referencia para toda la consulta.

Las cortesías cuentan como entradas emitidas aunque su importe sea cero.

Si dos películas tienen la misma cantidad vendida, se ordenan por su identificador.

Salida obtenida. La captura no contiene filas dentro de la semana consultada.



Reporte 3: Métodos de pago preferidos

SQL

```
-- R3. Porcentaje de uso por método de pago (pagos APROBADOS, últimos 720 h).
WITH corte AS (SELECT now() AS hasta, now() - interval '720 hours' AS desde),
por_metodo AS (
 SELECT p.metodo,
 count(DISTINCT p.id_compra) AS compras,
 sum(p.monto) AS monto
 FROM pago p CROSS JOIN corte k
 WHERE p.estado = 'Aprobado' -- rechazados no cuentan
 AND p.creado_en >= k.desde AND p.creado_en < k.hasta
 GROUP BY p.metodo
),
totales AS (SELECT sum(compras) AS t_compras, sum(monto) AS t_monto FROM por_metodo)
SELECT m.metodo,
 m.compras,
 round(100.0 * m.compras / nullif(t.t_compras,0), 2) AS pct_compras, -- denominador: compras
pagadas totales
 m.monto,
 round(100.0 * m.monto  / nullif(t.t_monto,0), 2) AS pct_monto, -- denominador: monto aprobado total
 'USD' AS moneda
FROM por_metodo m CROSS JOIN totales t
ORDER BY m.compras DESC, m.metodo;
```

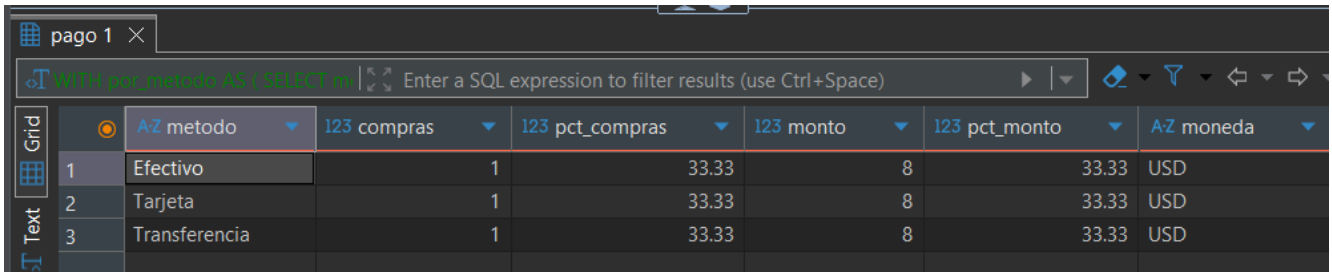
Explicación del resultado

Se calculan dos porcentajes. Uno compara la cantidad de compras y el otro compara los montos pagados.

La consulta evita dividir entre cero cuando no existen pagos en el periodo.

El método Cortesia puede aparecer con monto cero. En ese caso participa en el porcentaje de compras, mientras que su aporte al monto recaudado es cero.

Salida obtenida. La consulta de métodos de pago no devolvió filas para las últimas 720 horas. La imagen muestra las columnas del resultado obtenido en la base p2.



Reporte 4: Auditoría crítica del último mes por administrador

SQL

```
-- Auditoria Critica del ultimo mes
WITH corte AS (SELECT now() AS hasta, now() - interval '720 hours' AS desde)
SELECT a.id_usuario_actor AS id_administrador,
 u.nombre AS nombre_administrador,
 count(*) FILTER (WHERE a.accion NOT LIKE 'denegado:%') AS acciones_manuales,
```

```
 count(*) FILTER (WHERE a.accion = 'otorgar_cortesia') AS cortesias,
 count(*) FILTER (WHERE a.accion IN ('reembolso_ordinario','reembolso_forzado')) AS reembolsos,
 count(*) FILTER (WHERE a.accion = 'reembolso_forzado') AS reembolsos_forzados,
 count(*) FILTER (WHERE a.accion = 'rechazar_reembolso') AS reembolsos_rechazados,
 count(*) FILTER (WHERE a.accion = 'cambio_asiento') AS cambios_asiento,
 count(*) FILTER (WHERE a.accion = 'configurar_politica') AS cambios_politica,
 count(*) FILTER (WHERE a.accion ~ '^(insert|update|delete)_') AS cambios_catalogo_precios_roles,
 count(*) FILTER (WHERE a.accion LIKE 'denegado:%') AS intentos_denegados,
 max(a.fecha) AS ultima_accion
FROM admin_operacion_audit a
JOIN usuario u ON u.id_usuario = a.id_usuario_actor
CROSS JOIN corte k
WHERE a.rol_al_actuar = 'administrador'
  AND a.fecha >= k.desde AND a.fecha < k.hasta
GROUP BY a.id_usuario_actor, u.nombre
ORDER BY a.id_usuario_actor;
-- R4b. Detalle consultable de las mismas acciones (una fila por acción).
WITH corte AS (SELECT now() AS hasta, now() - interval '720 hours' AS desde)
SELECT a.id_usuario_actor AS id_administrador, a.fecha, a.accion, a.entidad, a.id_entidad,
 a.ip, a.motivo, a.cambios
FROM admin_operacion_audit a CROSS JOIN corte k
WHERE a.rol_al_actuar = 'administrador'
  AND a.accion NOT LIKE 'denegado:%'
  AND a.fecha >= k.desde AND a.fecha < k.hasta
ORDER BY a.id_usuario_actor, a.fecha DESC, a.id_admin_operacion_audit DESC;
```

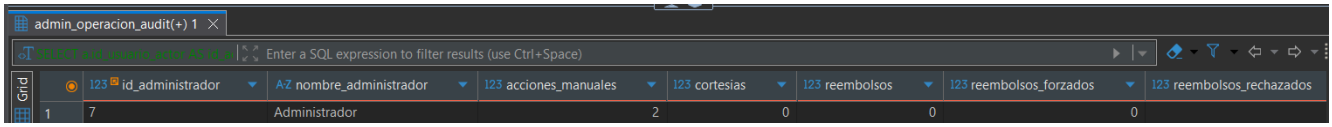
Explicación del resultado

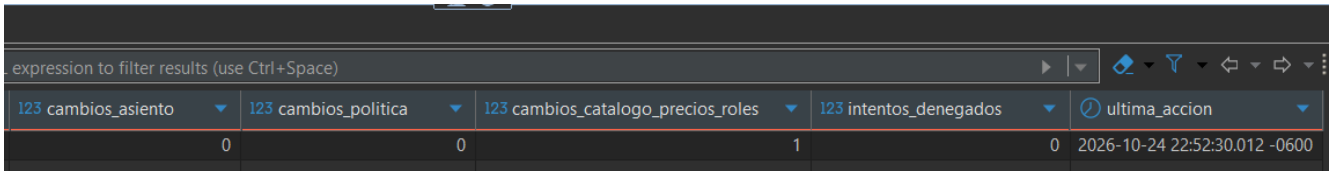
El resultado separa las acciones manuales, las cortesías y los reembolsos de cada administrador.

Los intentos rechazados aparecen aparte y no se cuentan como cambios completados.

Los triggers registran las inserciones, modificaciones y borrados de las tablas controladas.

Salida obtenida. El resumen devuelve un administrador con dos acciones manuales. Una corresponde a cambios de catálogo, precios o roles. No registra cortesías, reembolsos ni intentos denegados en el periodo consultado. La fila se presenta con los campos en vertical para facilitar su lectura.





Implementación de pruebas y resultados

Entorno y aislamiento

Las pruebas conservadas se ejecutaron con Python 3.14.7, PostgreSQL 18.6, Flask 3.1.3 y psycopg 3.3.6. Cada suite utiliza un espacio temporal de la base para no mezclar sus datos con los del proyecto.

Las capturas muestran compras y devoluciones simuladas en un entorno separado. La evaluación reúne los resultados de 97 pruebas funcionales y de integración registradas durante el desarrollo.

Suite	Pruebas aprobadas	Alcance
verificar_a02	11	Catálogo, lotes, zona horaria y programación.
verificar_cartelera_a03	5	Funciones futuras/activas y filtros.
verificar_precios_a03	11	Tarifas, condiciones y vigencias.
verificar_seguridad_a	15	Sesión, CSRF, permisos y auditoría.
verificar_compatibilidad_c03	5	Compatibilidad de mecanismos de C.
verificar_salas_a04	17	Protección y coordinación de distribución.
verificar_integracion_b_a04	10	Reserva y pago reales de B con A.
verificar_integracion_global	23	Flujo completo de A/B/C.
Total	97	Ocho salidas aprobadas.

Evidencia y reproducción

Las versiones previas del repositorio conservan las salidas de las pruebas realizadas durante el desarrollo. Las pruebas del flujo integrado se incluyen en dev/pruebas para repetir los casos principales.

También se revisó que la actualización de la base conservara datos y contraseñas. Esa comprobación no sustituye una recuperación después de un desastre ni una instalación en otra computadora.

Casos de integridad y concurrencia

Caso	Precondición y operación	Resultado registrado
Horarios y limpieza	Función válida; insertar un cruce o dejar menos de 15 minutos.	Rechazo; límite exacto permitido.
Lote duplicado	Un asiento ya existe dentro del lote solicitado.	Reversión completa sin filas parciales.
Tarifa ambigua	Vigencias superpuestas para la misma combinación.	Rechazo, incluso con dos altas competidoras.
Promoción	Dos entradas a USD 8 en miércoles y jueves elegibles.	Miércoles USD 8; jueves USD 16.
Cotización histórica	Reservar y cambiar luego la tarifa de catálogo.	Pago conserva el precio reservado.
Pago y vencimiento	Rechazo, expiración o pago tardío.	No se emite boleto y se libera según estado.
QR y acreditación	Boleto correcto, repetido, de otra función o sin credencial.	Un ingreso válido; demás intentos rechazados.
Devolución parcial	Compra con dos boletos sin usar; devolver uno.	Anula y libera solo el elegido, preserva historia.
Auditoría fallida	Provocar fallo al registrar una cortesía o cambio.	Reversión de la operación de negocio.
Propiedad y permisos	Consultar QR ajeno o enviar formulario sin permiso/CSRF.	Denegación del servidor.

Carreras comprobadas

Cuando 50 solicitudes intentaron reservar el mismo asiento, una tuvo éxito y las otras 49 se rechazaron. Al leer el mismo QR 50 veces se aceptó un solo ingreso y se guardaron los 50 intentos.

También se probaron operaciones que competían entre sí, como reservar mientras se intentaba borrar un asiento, o ingresar con un boleto mientras se solicitaba devolverlo. La base mantuvo un estado válido y evitó referencias rotas.

Las pruebas comprueban las reglas de negocio y los procesos de la aplicación. Incluyen la consulta de cartelera, la compra, el ingreso, la devolución y la administración. Su alcance corresponde al prototipo local y a los casos de concurrencia evaluados.

Rendimiento y consultas pesadas

Estado de la evidencia

EXPLAIN ANALYZE muestra qué pasos realiza PostgreSQL para resolver una consulta y cuánto tarda. Comparamos cinco consultas antes y después de crear índices de apoyo. Los resultados corresponden a una base de prueba con pocos registros y varias consultas no encontraron filas.

Los índices de claves primarias y restricciones ya existían en la primera medición. La segunda agrega índices y actualiza las estadísticas con ANALYZE. Los tiempos permiten describir ese ensayo, pero no separar por sí solos el efecto del índice, las estadísticas y los datos que ya estaban en memoria.

Consultas evaluadas

```
-- Q1: ocupación/recaudación de UNA función
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(b.id_boleto) FILTER (WHERE b.estado IN ('Emitido','Usado')) vendidos,
 sum(cd.total_linea) bruto
FROM compra c
JOIN compra_detalle cd ON cd.id_compra=c.id_compra
LEFT JOIN boleto b ON b.id_compra=cd.id_compra AND b.id_asiento=cd.id_asiento
WHERE c.id_funcion = 5000 AND c.estado IN ('Pagada','ParcialmenteReembolsada','Reembolsada');
-- Q2: top 5 de la semana
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.id_pelicula, count(*) vendidos
FROM boleto b
JOIN compra c ON c.id_compra=b.id_compra
JOIN funcion f ON f.id_funcion=c.id_funcion
WHERE b.emitido_en >= now()-interval '168 hours' AND b.emitido_en < now()
  AND b.estado IN ('Emitido','Usado') AND c.estado IN ('Pagada','ParcialmenteReembolsada')
GROUP BY f.id_pelicula ORDER BY vendidos DESC, f.id_pelicula LIMIT 5;
-- Q3: métodos de pago (30 días)
EXPLAIN (ANALYZE, BUFFERS)
SELECT metodo, count(DISTINCT id_compra), sum(monto)
FROM pago WHERE estado='Aprobado' AND creado_en >= now()-interval '720 hours' AND
creado_en < now()
GROUP BY metodo;
-- Q4: auditoría por administrador (30 días)
EXPLAIN (ANALYZE, BUFFERS)
SELECT id_usuario_actor, count(*)
FROM admin_operacion_audit
WHERE rol_al_actuar='administrador' AND fecha >= now()-interval '720 hours' AND fecha < now()
GROUP BY id_usuario_actor;
-- Q5: disponibilidad de asientos de una función
EXPLAIN (ANALYZE, BUFFERS)
SELECT id_asiento, estado FROM inventario_funcion WHERE id_funcion = 5000;
```

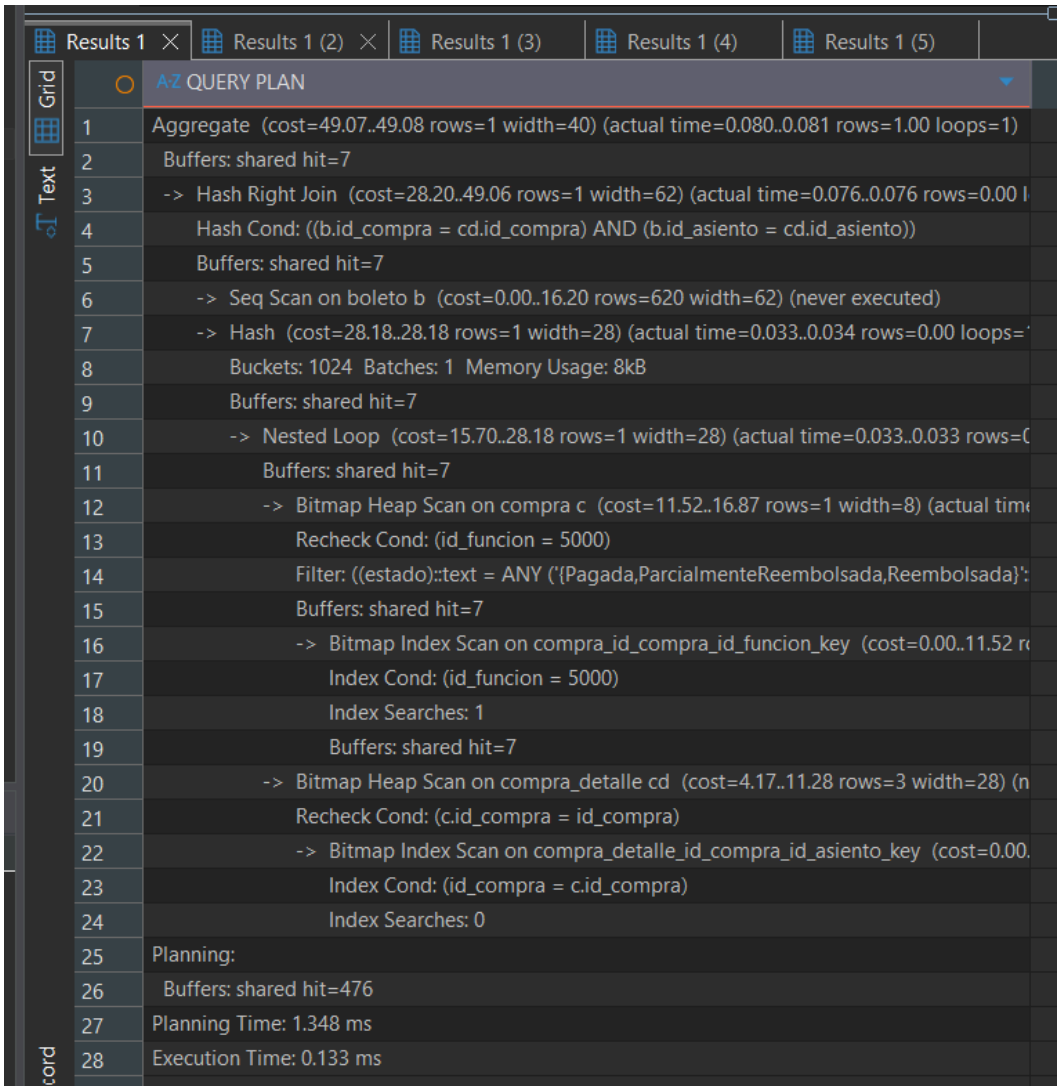
Índices adicionales

```
CREATE INDEX idx_compra_funcion ON compra (id_funcion); -- Q1
CREATE INDEX idx_boleto_compra_asi ON boleto (id_compra, id_asiento); -- Q1 (el join por compra
no tenía índice)
CREATE INDEX idx_boleto_emitido ON boleto (emitido_en) INCLUDE (estado, id_compra);  -- Q2
CREATE INDEX idx_pago_aprobado_fecha ON pago (creado_en) INCLUDE (metodo, id_compra,
monto)
 WHERE estado='Aprobado'; -- Q3 (índice parcial)
CREATE INDEX idx_audit_admin_fecha ON admin_operacion_audit (fecha, id_usuario_actor)
 WHERE rol_al_actuar='administrador'; -- Q4
ANALYZE;
```

Antes de los índices adicionales

La primera medición conserva los índices creados automáticamente por las claves. Por eso se describe como antes de los índices adicionales y no como una base sin ningún índice.

Consulta Q1



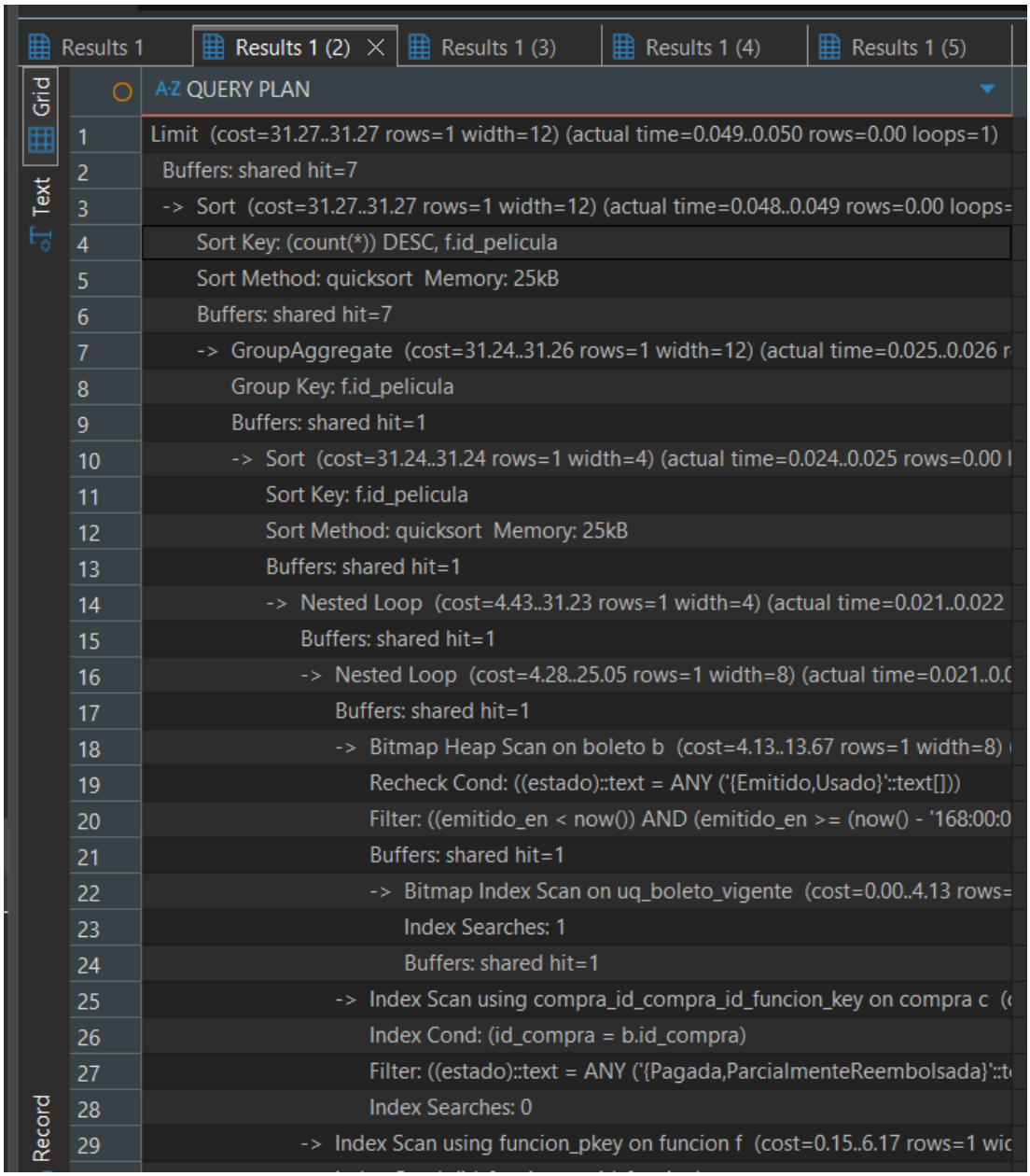
Análisis de rendimiento:

La consulta Q1 usa un índice existente de compra para buscar la función 5000. No encuentra compras que permitan continuar las uniones. El paso sobre boleto aparece como nunca ejecutado.

La planificación tardó 1,348 ms y la ejecución 0,133 ms.

Ese resultado permite leer el plan de una búsqueda sin coincidencias. No mide el costo de sumar una función con muchas ventas.

Consulta Q2



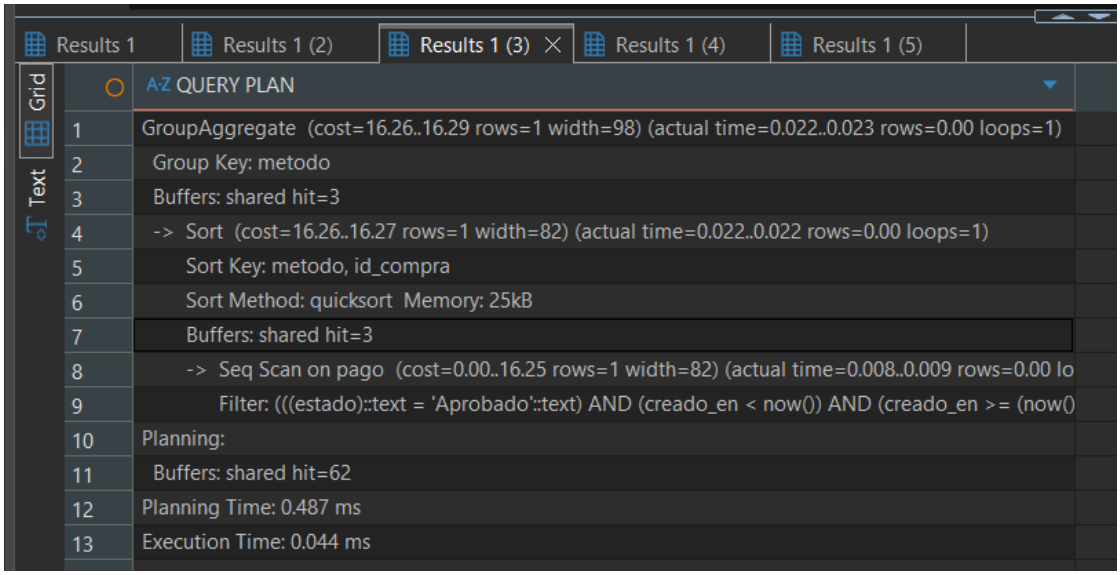
Análisis de rendimiento:

La consulta Q2 utiliza el índice parcial uq_boleto_vigente para revisar boletos vigentes. La fecha de emisión se aplica como un filtro adicional y no como una condición de búsqueda de ese índice.

La planificación tardó 0,837 ms y la ejecución 0,144 ms.

El resultado no contiene ventas del periodo. Por eso no permite evaluar el trabajo de las uniones con una semana de datos representativos.

Consulta Q3



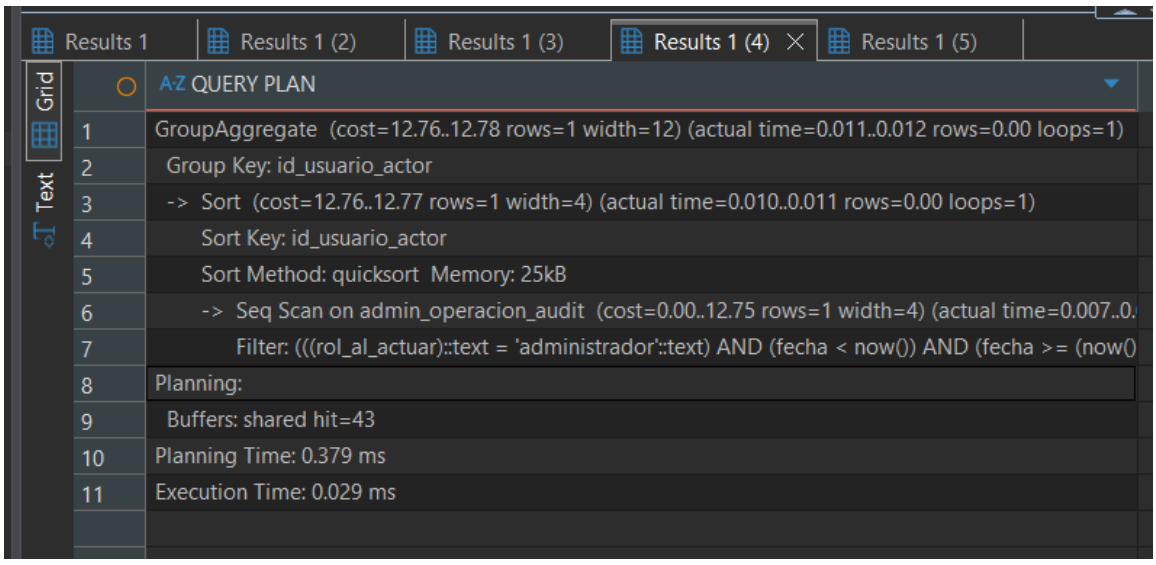
Análisis de rendimiento:

La consulta Q3 recorre la tabla pago y filtra los pagos aprobados del periodo. Este recorrido completo se llama Seq Scan.

La ejecución tardó 0,044 ms.

Con pocos datos, revisar la tabla completa puede ser una decisión razonable del motor.

Consulta Q4



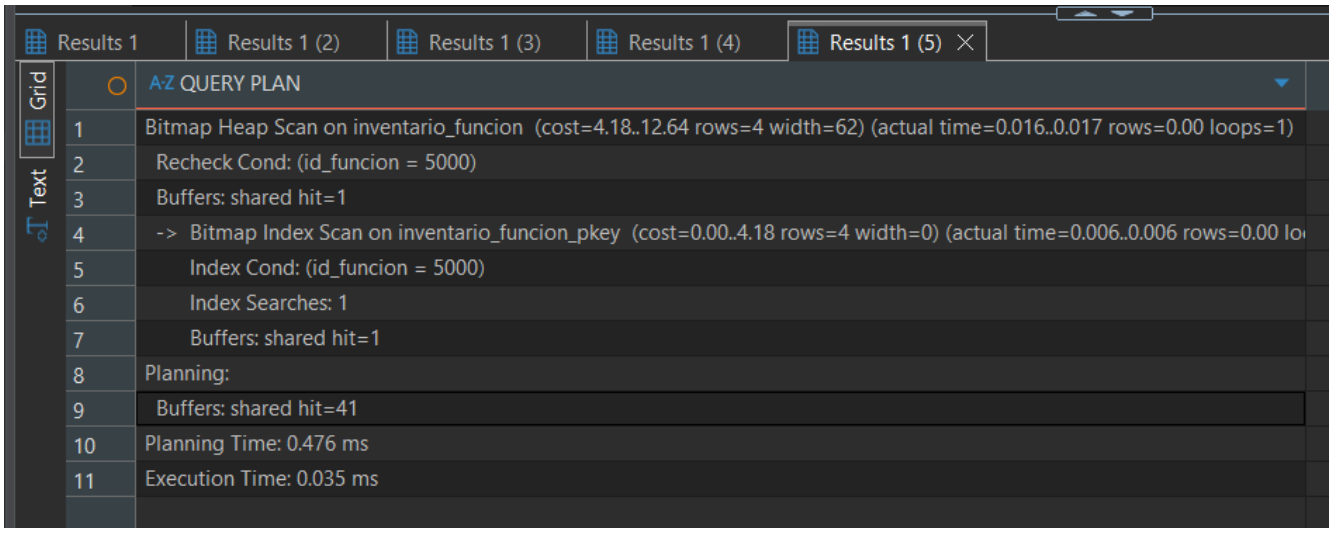
Análisis de rendimiento:

La consulta Q4 también usa Seq Scan. Revisa las acciones de administradores y el rango de fechas antes de agruparlas.

La ejecución tardó 0,029 ms y no produjo filas.

El índice propuesto organiza las acciones de administradores por fecha. En la ejecución mostrada PostgreSQL eligió recorrer la tabla completa porque el volumen de datos es pequeño.

Consulta Q5



Análisis de rendimiento:

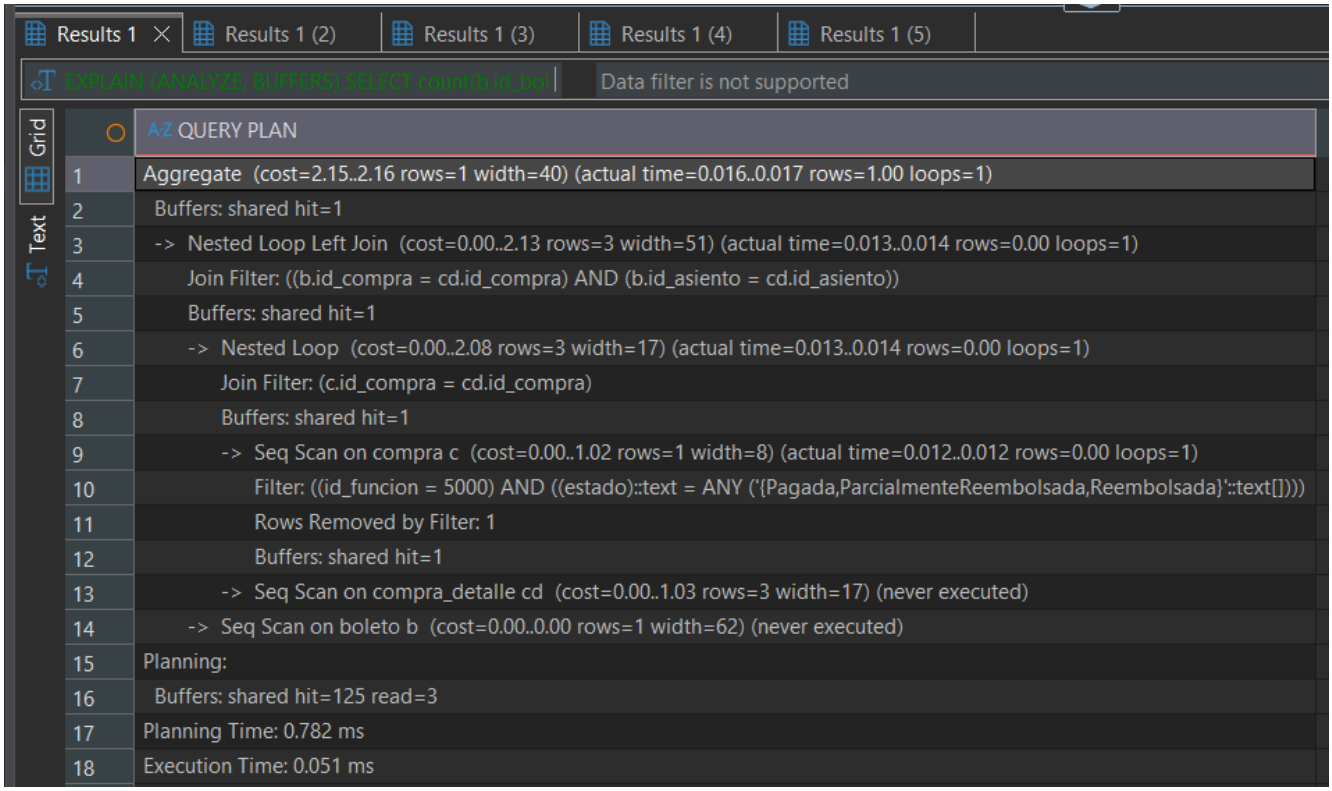
La consulta Q5 utiliza la clave primaria de inventario_funcion para buscar los asientos de la función 5000 mediante un Bitmap Index Scan.

La planificación tardó 0,476 ms y la ejecución 0,035 ms.

La búsqueda no devuelve asientos para esa función. El plan muestra cómo se usa la clave, pero no representa una consulta de disponibilidad con inventario vendido o reservado.

Después de los índices adicionales

Consulta Q1

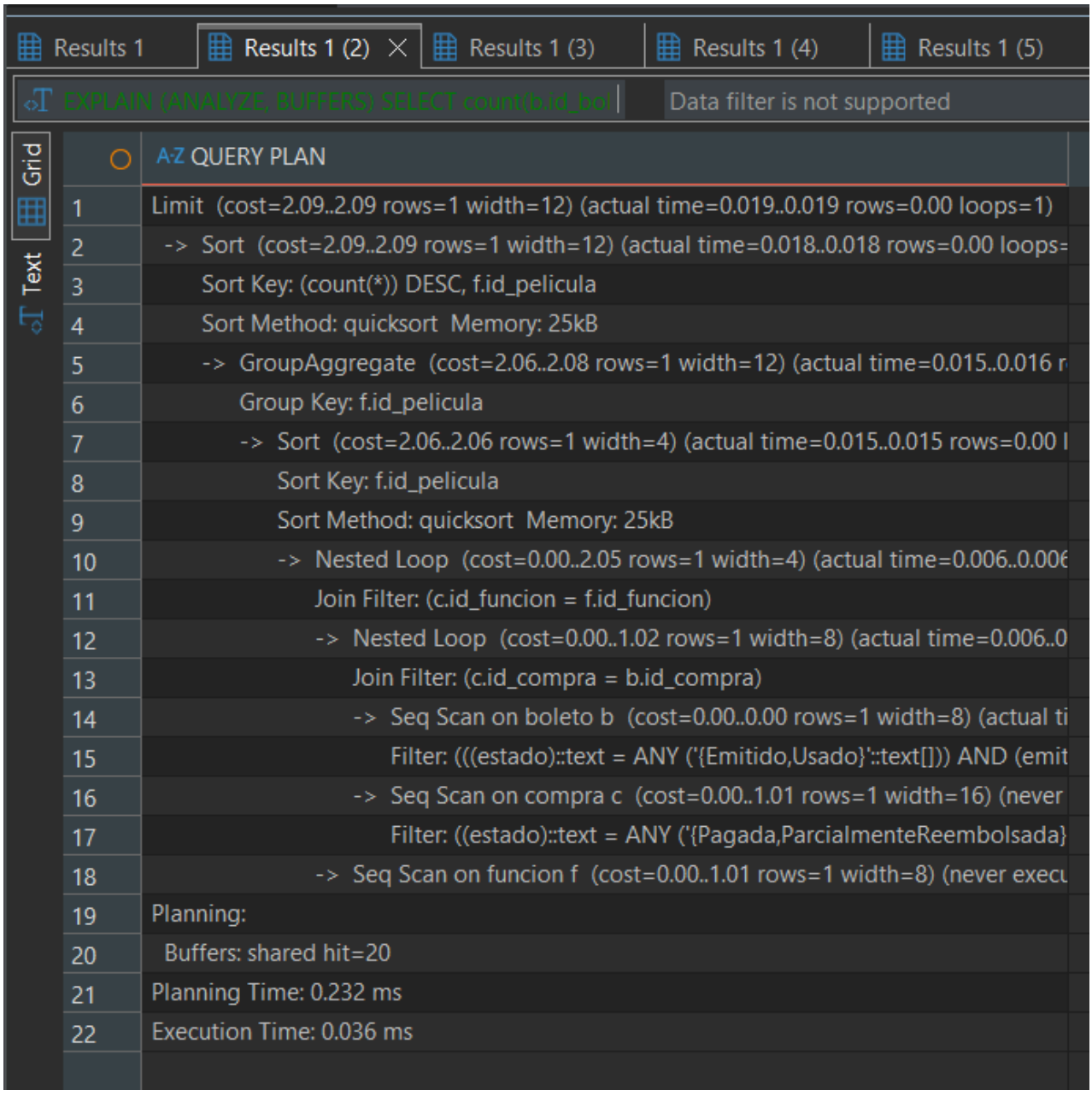


Análisis de rendimiento

Después de agregar índices y actualizar las estadísticas, Q1 utiliza recorridos completos y uniones de tipo Nested Loop. La captura no muestra el uso de los nuevos índices. Algunas partes del plan no llegaron a ejecutarse.

La ejecución pasó de 0,133 a 0,051 ms. No podemos atribuir esta diferencia únicamente a los índices.

Consulta Q2

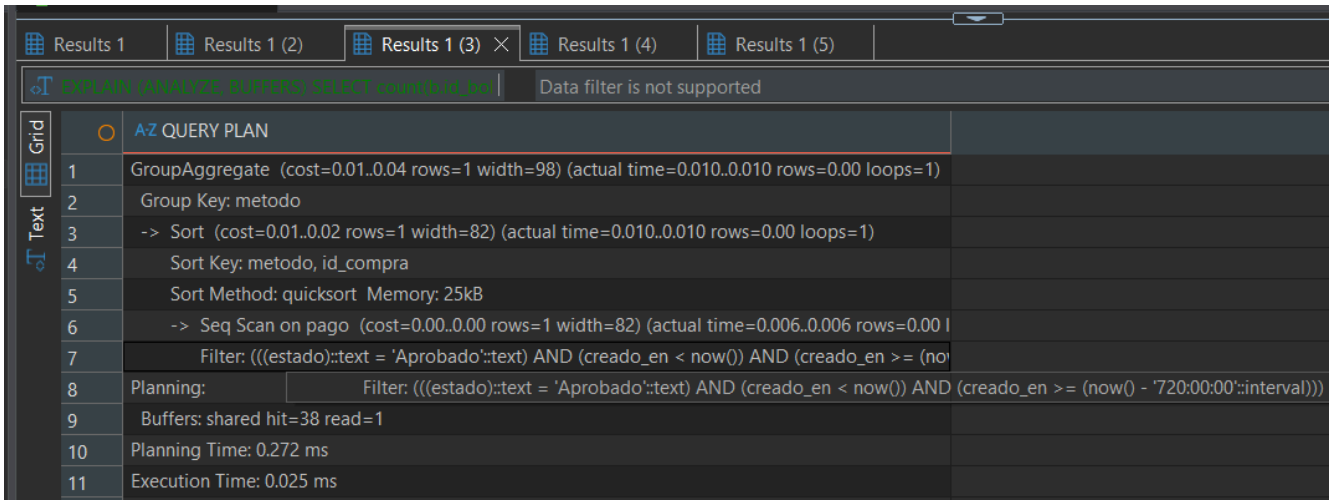


Análisis de rendimiento

En Q2 el nuevo plan recorre la tabla boleto con Seq Scan. La captura no muestra el uso de idx_boleto_emitido.

La planificación pasó de 0,837 a 0,232 ms y la ejecución de 0,144 a 0,036 ms. El resultado sigue vacío.

Consulta Q3

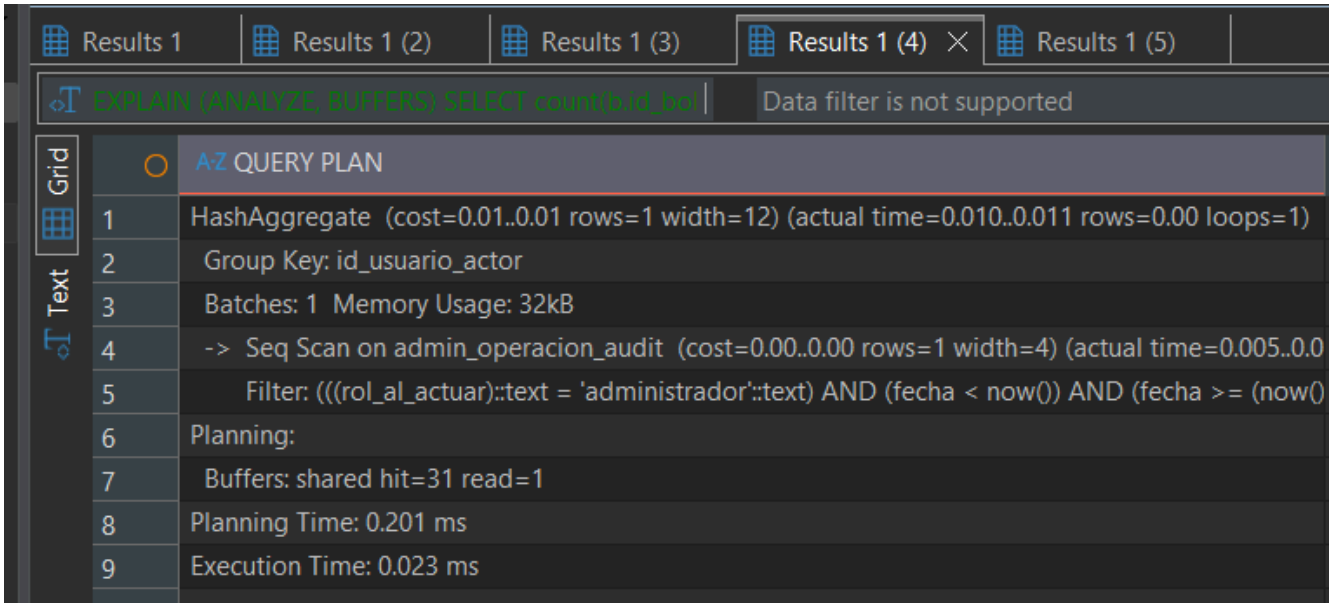


Análisis de rendimiento

Q3 mantiene el recorrido completo de pago. El índice parcial se creó, pero el plan mostrado no lo utiliza. Su presencia no garantiza que el tiempo se mantenga igual cuando crezcan los datos.

La ejecución pasó de 0,044 a 0,025 ms.

Consulta Q4

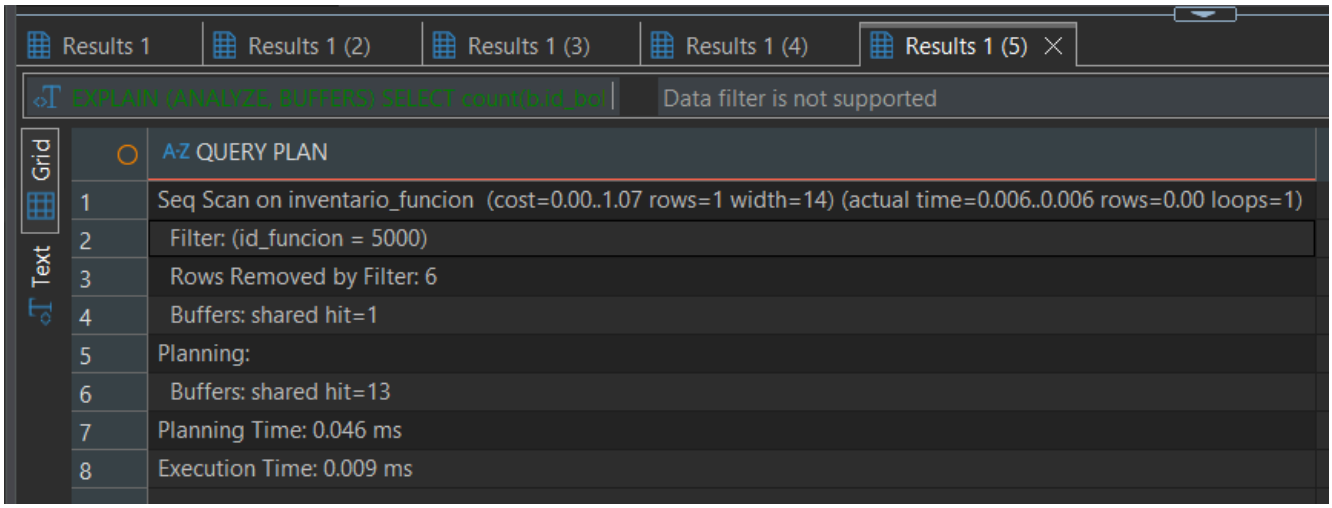


Análisis de rendimiento

Q4 recorre la tabla completa y agrupa en memoria. El plan mostrado utiliza Seq Scan y HashAggregate.

La ejecución pasó de 0,029 a 0,023 ms.

Consulta Q5



Análisis de rendimiento

Q5 pasa de un recorrido por índice a un Seq Scan. La captura muestra seis filas descartadas por el filtro y ningún asiento de la función 5000.

La ejecución pasó de 0,035 a 0,009 ms. La lectura fue corta, pero ese resultado no demuestra una mejora por los índices adicionales.

Resumen de los tiempos de ejecución

Consulta	Antes en ms	Después en ms	Plan posterior
Q1 Ventas por función	0,133	0,051	Seq Scan y Nested Loop
Q2 Top de películas	0,144	0,036	Seq Scan y agregación
Q3 Métodos de pago	0,044	0,025	Seq Scan y GroupAggregate
Q4 Auditoría	0,029	0,023	Seq Scan y HashAggregate
Q5 Disponibilidad	0,035	0,009	Seq Scan

Alcance de la comparación

La tabla compara los tiempos registrados antes y después de agregar los índices y actualizar las estadísticas. En los planes posteriores PostgreSQL eligió recorridos completos de las tablas.

Estas mediciones describen las consultas ejecutadas con el volumen de datos de la prueba. La diferencia de tiempos no demuestra por sí sola una mejora causada por los índices.

Operación y mantenimiento

El programa se inicia con iniciar.py o iniciar.cmd. El iniciador usa el entorno virtual y pone en marcha la revisión de reservas vencidas. Para publicar la aplicación fuera del equipo local se necesitaría preparar un servidor adecuado y revisar la red y los permisos.

Continuidad respaldos y privacidad

Objetivos y recuperación

La propuesta establece una pérdida máxima de cinco segundos de datos recientes y la recuperación del servicio crítico en menos de cinco minutos. Estos valores son objetivos de diseño, distintos de una medición del prototipo local. Una restauración completa o la recuperación de un borrado tienen un alcance mayor.

Una réplica ayuda a continuar el servicio y un respaldo permite volver a un estado anterior. Si se borra información por error, la réplica también puede recibir el borrado. La conmutación contempla un único servidor habilitado para aceptar cambios y así evita escrituras simultáneas en dos instancias.

Mecanismo propuesto	Frecuencia o retención	Propósito y límite
WAL continuo	Retención de 30 días	Recuperación a un instante; pérdida depende de transmisión real.
Respaldo base	Semanal, cinco completos	Punto de partida para restaurar.
Incremental	Diario, 30 días	Reducir trabajo de restauración; requiere cadena válida.
Volcado lógico	Diario para esquema y catálogos	Portabilidad; no proporciona PITR.
Archivo de auditoría	Retención mínima de 60 meses, dos copias	Conserva trazas de operaciones; no reemplaza el respaldo de la base.
Verificación	Semanal y simulacros periódicos propuestos	Restaurar y comprobar integridad y tiempos.

La continuidad se presenta como una propuesta de operación. Su alcance comprende la pérdida de un servidor o región y la conservación de una segunda copia de los respaldos.

Auditoría y seguridad

El sistema mantiene cuatro bitácoras para pagos, inventario, puerta y administración. Permiten conocer qué ocurrió y quién hizo un cambio. Los permisos de acceso limitan las operaciones que corresponden a cada rol.

Para conservar cinco años de auditoría se propone archivar copias protegidas y controlar su integridad y sus accesos. El archivo inmutable y la organización por periodos forman parte de la propuesta para una operación de mayor tamaño.

Las contraseñas se guardan mediante hashes y las claves de conexión quedan fuera del repositorio. La propuesta de privacidad contempla acceso según el rol y conservación limitada de los datos personales necesarios para las operaciones del cine.

Conclusiones

El proyecto muestra que la disponibilidad pertenece a un asiento dentro de una función. La reserva, el pago y el boleto necesitan mantenerse coordinados para que una venta sea válida de principio a fin.

La aplicación integra catálogo, precios, compra, ingreso, devolución y auditoría. Las restricciones y las 97 pruebas registradas respaldan los casos revisados. Las capturas permiten seguir el funcionamiento sin depender solo de leer el código.

Los diagramas ayudan a comprender la organización de los datos y su relación con las operaciones del programa. La comparación de consultas permite observar cómo PostgreSQL busca y agrupa la información durante las pruebas.

El proyecto reúne la base de datos y la interfaz en una solución que permite seguir una venta desde la selección de asientos hasta el ingreso a la sala. Los registros de auditoría y los reportes permiten consultar las operaciones realizadas y conservar su historial.

Evidencias visuales

Estas imágenes muestran una demostración de la aplicación con cuentas y datos de prueba. Se compraron dos entradas Adulto en zona General para una función del miércoles 4 de noviembre de 2026. Cada entrada costaba USD 8 y el 2x1 dejó un total de USD 8. Los pagos son simulados.

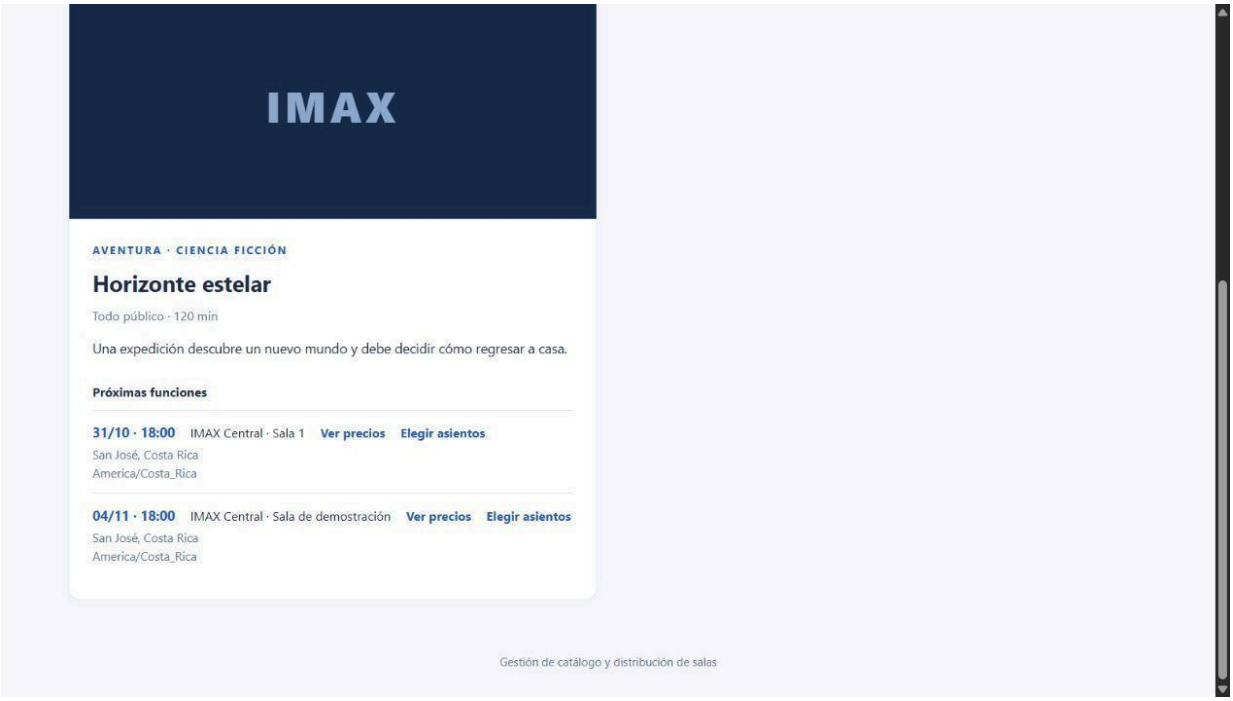


Figura 4 Cartelera con metadatos, ubicación y horarios. El ensayo muestra el respaldo IMAX al no asignar un póster externo.

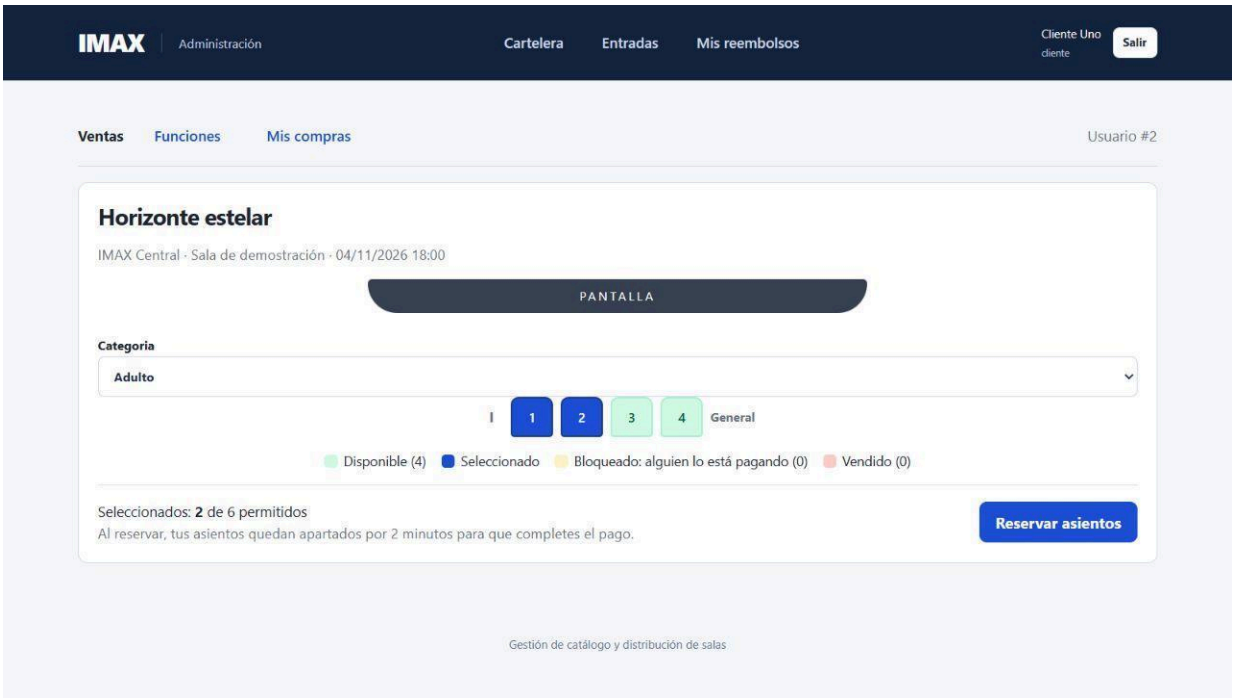


Figura 5 Selección de I1 e I2 y límite de seis asientos por compra.

Reserva y entrega digital

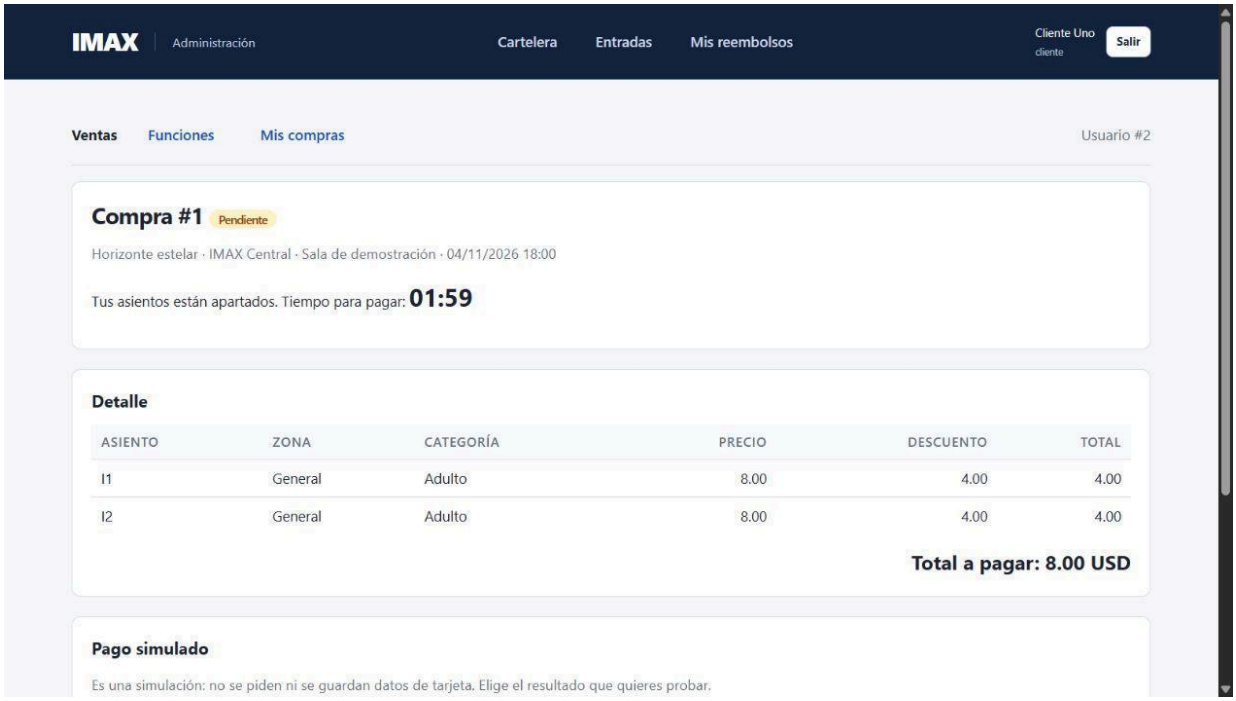


Figura 6 Reserva vigente con temporizador de 01:59, descuento de USD 4 por entrada y total USD 8.

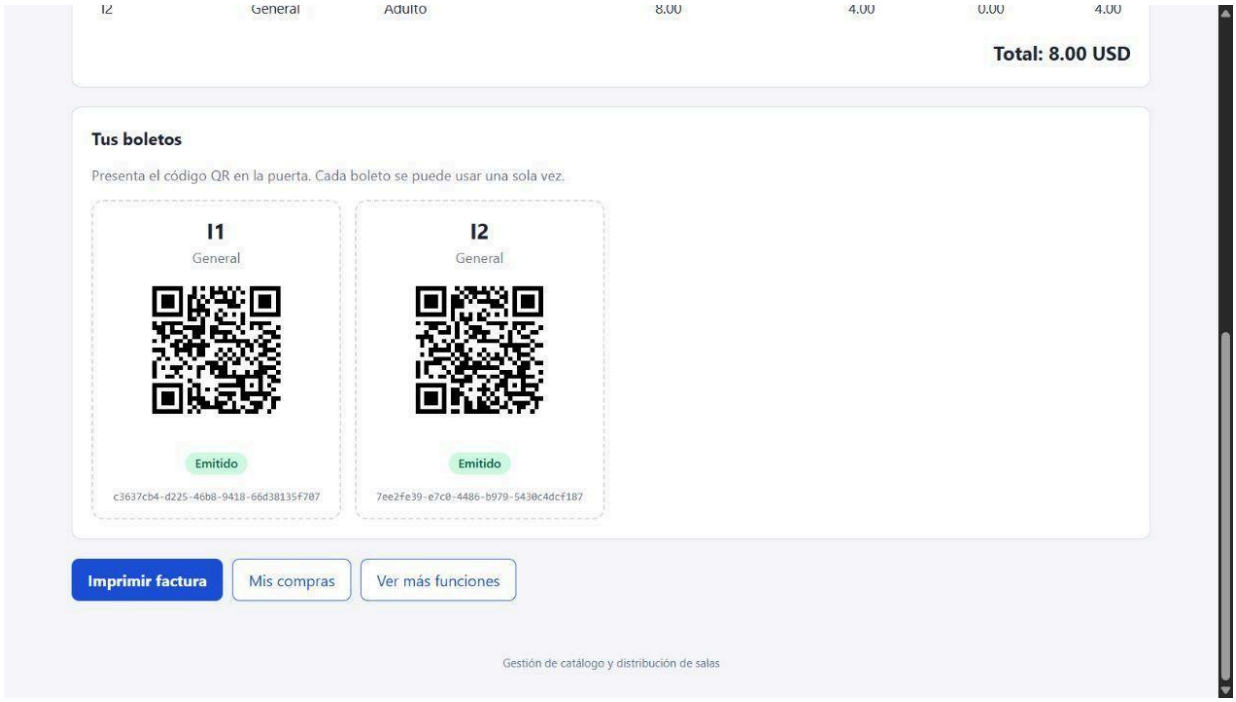


Figura 7 Tras aprobar el pago simulado se emiten dos boletos, cada uno con su propio QR.

Reportes y devolución

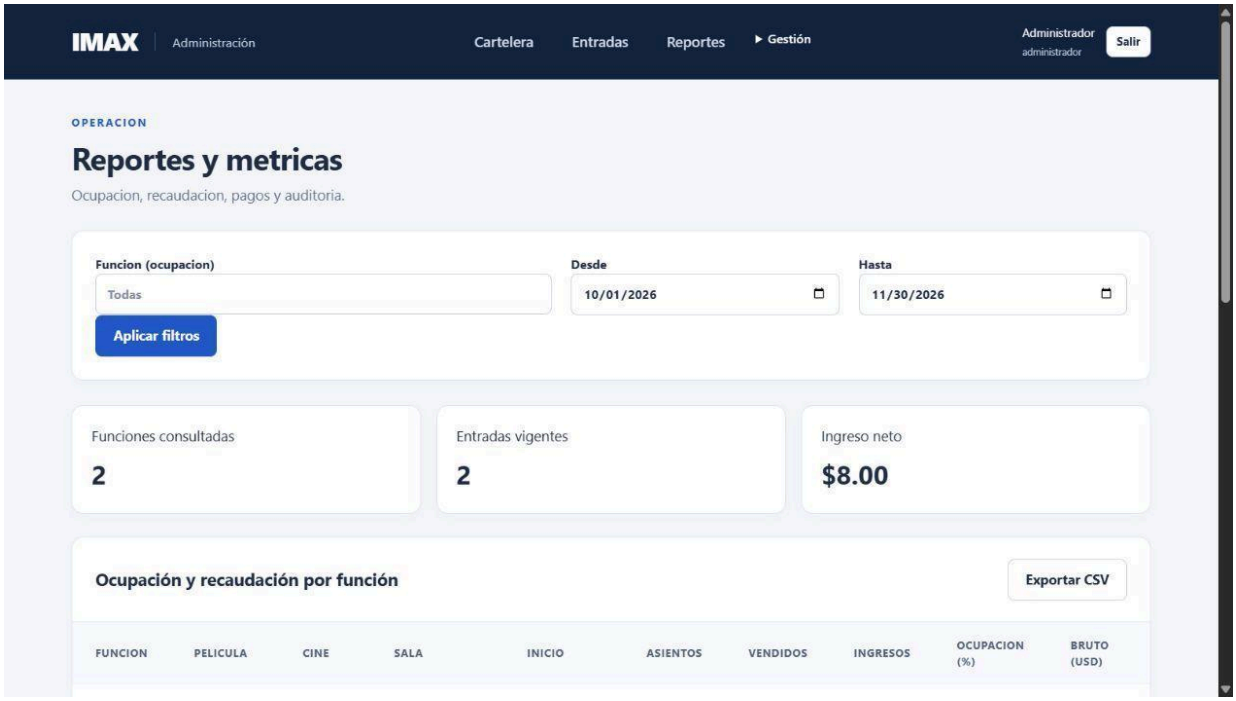


Figura 8 Antes del reembolso: periodo visible, dos entradas vigentes e ingreso neto USD 8.

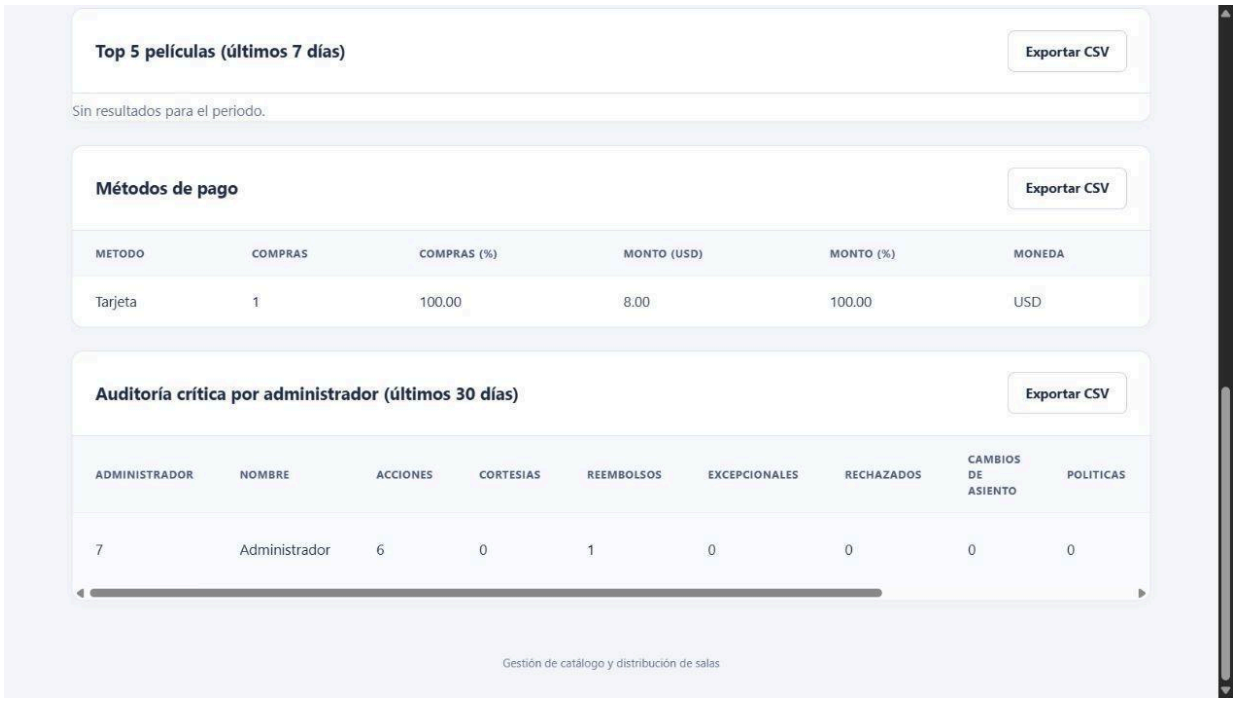


Figura 9 Después de devolver la compra: el resumen registra un reembolso del administrador. El pago original conserva su historia.

En esta demostración se devolvieron los USD 8 completos y el ingreso neto quedó en cero. La devolución parcial se probó por separado. En ese caso solo se anula el boleto elegido y el otro permanece vigente.

Ingreso único a sala

El ingreso se probó con otro boleto de la función 4, dentro del horario permitido. No corresponde a los boletos ya devueltos. El operador leyó el mismo código dos veces.

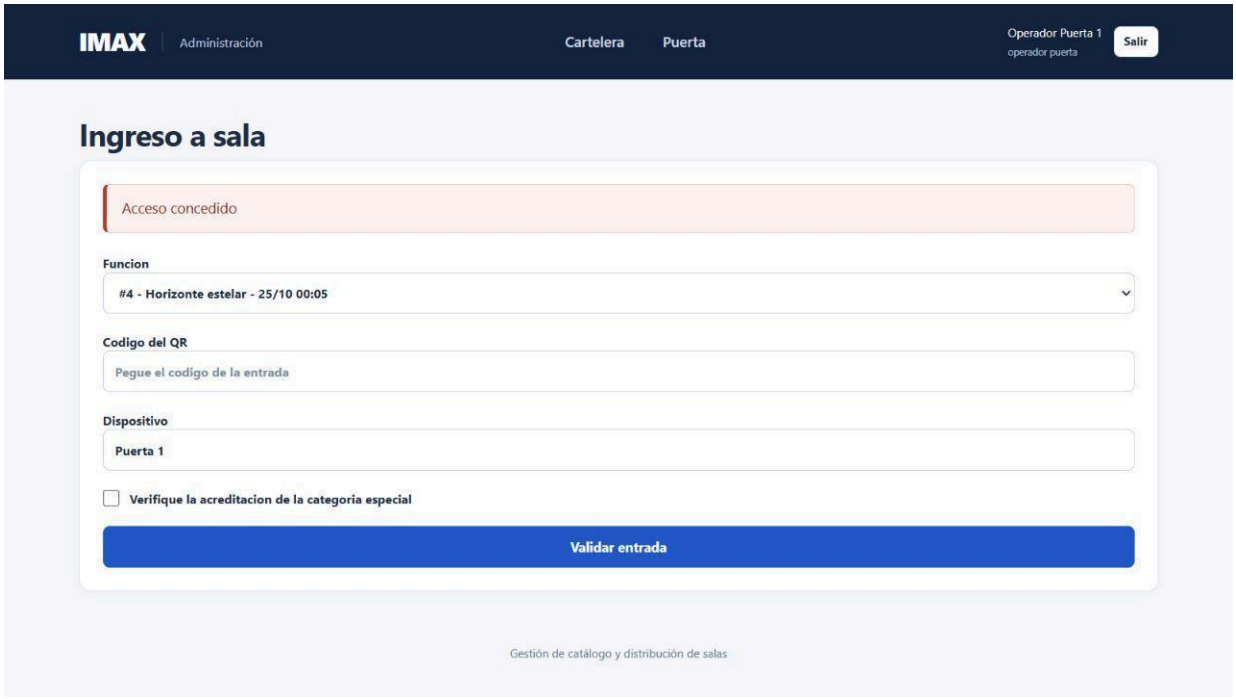


Figura 10 Primer intento con el boleto de prueba: acceso concedido.

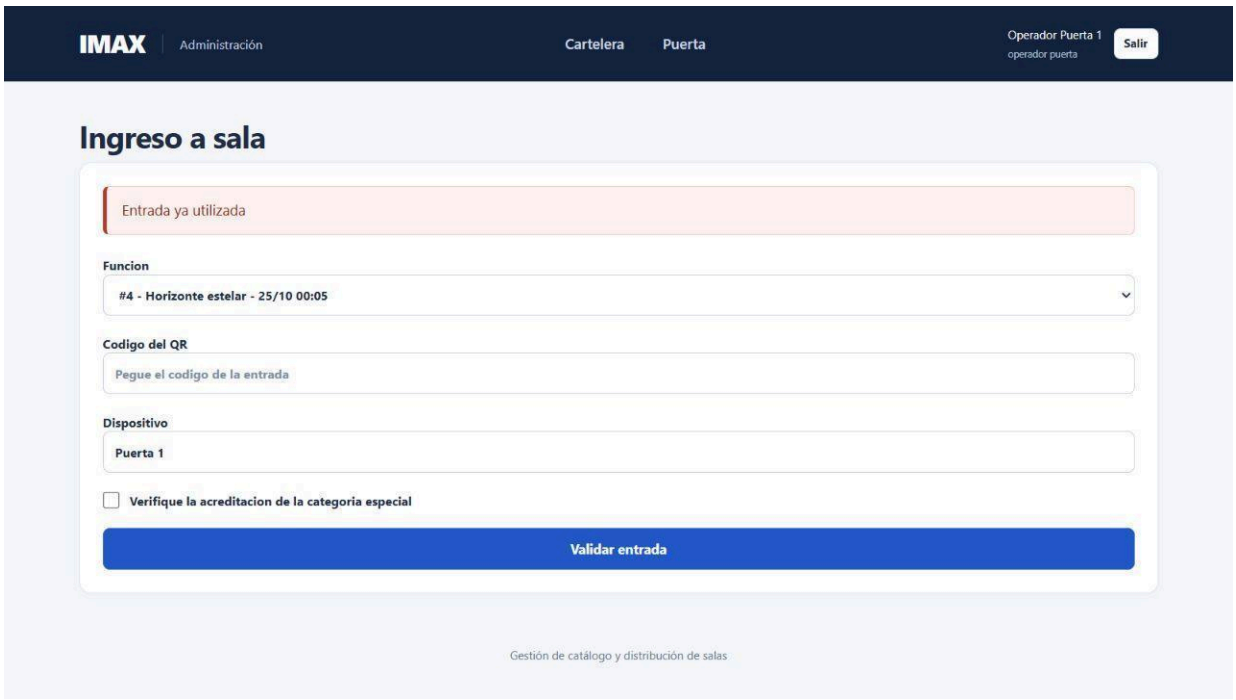


Figura 11 Segundo intento con el mismo boleto: entrada ya utilizada.

Conservación del historial

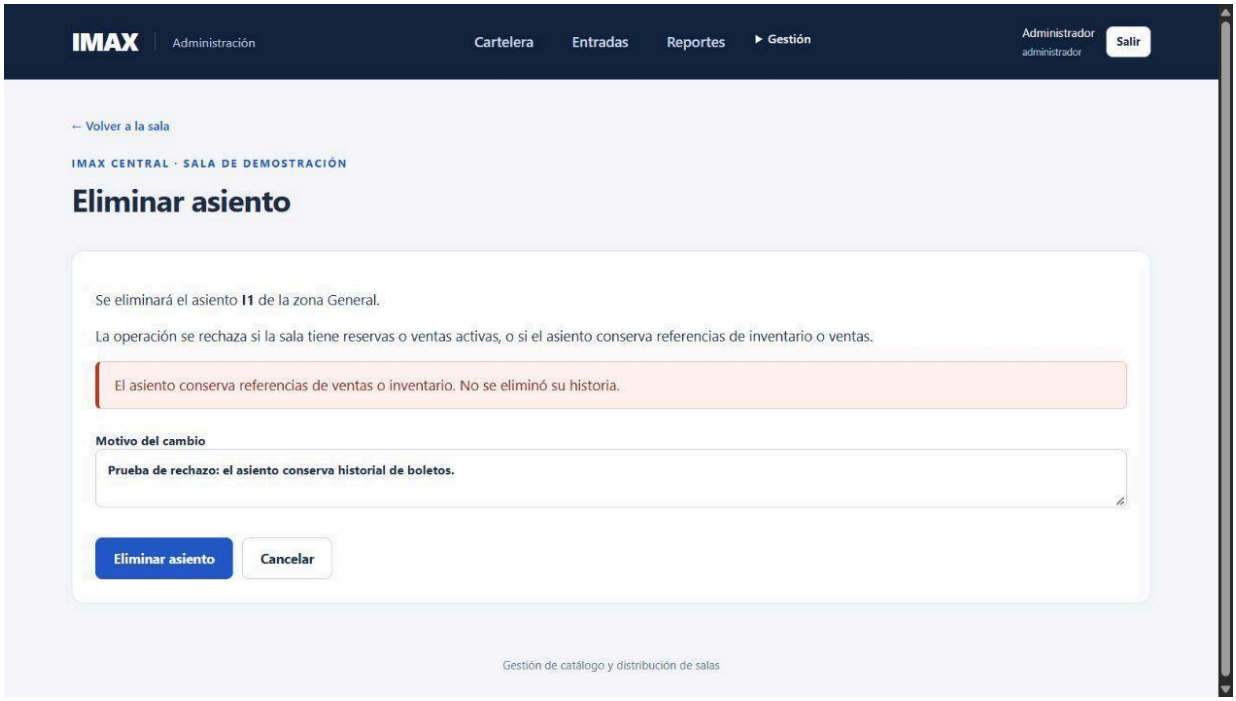


Figura 12 El intento de eliminar I1 se rechaza porque el asiento conserva referencias de ventas o inventario.

La devolución libera el asiento y anula el boleto correspondiente sin borrar la compra anterior. Esto permite explicar lo ocurrido aunque el asiento vuelva a venderse.

Referencias

https://flask.palletsprojects.com/en/stable/

https://flask.palletsprojects.com/en/stable/deploying/

Anexo de uso de inteligencia artificial en la interfaz

Se utilizó inteligencia artificial como apoyo para construir y mejorar la interfaz web del proyecto. Ayudó a proponer la organización de las pantallas, los formularios, la navegación y los mensajes que recibe el usuario al realizar una operación.